

POLITECNICO MILANO 1863 | SCUOLA DI INGEGNERIA INDUSTRIALE E DELL'INFORMAZIONE

Master's Degree in Computer Science and Engineering

Project report

"Design Document"

"Best Bike Paths"

Academic Year: 2025/2026

Authors: **Daniele Toniolo** **([redacted])**
Stefano Ungaro **([redacted])**
Matteo Zappa **([redacted])**

Professor: **Matteo Camilli**

Contents

Table of Contents **II**

List of Figures **III**

List of Tables **IV**

1 Introduction **1**
1.1 Scope . . . 1
1.2 Definitions, Acronyms, Abbreviations . . . 1
1.2.1 Definitions . . . 1
1.2.2 Acronyms . . . 2
1.2.3 Abbreviations . . . 2
1.3 Revision History . . . 3
1.4 Reference Documents . . . 3
1.5 Document Structure . . . 3

2 Architectural Design **4**
2.1 Overview . . . 4
2.2 Component View . . . 4
2.2.1 Components Diagrams . . . 5
2.2.2 Components Descriptions . . . 5
2.3 Deployment View . . . 7
2.3.1 Overview . . . 7
2.3.2 Deployment Diagram . . . 7
2.3.3 Node mapping . . . 8
2.3.4 Detailed Layer Decisions . . . 9
2.4 Component Interfaces . . . 9
2.4.1 REST API Interface . . . 9
2.4.2 Public API Endpoints (Gateway) . . . 10
Internal Microservices Interfaces . . . 13
2.4.3 External Service Interfaces . . . 13
2.5 Runtime View . . . 14
2.6 Selected Architectural Styles and Patterns . . . 26
2.7 Other Design Decisions . . . 27

3 User Interface Design **28**
3.1 General Overview . . . 28
3.2 User Interface Layouts . . . 28
3.2.1 Landing Screen . . . 28
3.2.2 Explore . . . 29
3.2.3 Profile . . . 31
3.2.4 My Activity . . . 31

4 Requirements Traceability **34**
4.1 Functional Requirements Traceability . . . 34
4.2 Non-Functional Requirements Traceability . . . 36
4.2.1 Performance Requirements . . . 36
4.2.2 Software System Attributes . . . 37

5 Implementation, Integration and Test Plan **38**
5.1 Overview . . . 38
5.2 Implementation Plan . . . 38
5.3 Integration Strategy . . . 39
5.4 Test Plan . . . 40

6 Effort Spent **42**

7 References **43**
7.1 Used Tools . . . 43

A Detailed Diagrams **44**

List of Figures

2.1 UML Components Diagram. . . . 5
2.2 UML Deployment Diagram. . . . 7
2.3 SD1. Sign Up by a User 15
2.4 SD2. Log In by a User 16
2.5 SD3. Access as Guest 17
2.6 SD4. Record a Trip 19
2.7 SD5. Validate a Trip and a Path 20
2.8 SD6. Change Path's visibility 21
2.9 SD7. Create a Path Manually 22
2.10 SD8. Search for a Route 23
2.11 SD9. Delete a Trip 24
2.12 SD10. Delete a Path 25

3.1 Landing, Sign Up, and Sign In Screens 28
3.2 Explore Section Screens 29
3.3 Path Search Screens 29
3.4 Mockups of Recording Screen 30
3.5 Profile Screen 31
3.6 My Activity Screen 31
3.7 Trip Details Screens 32
3.8 Path Details Screen 32
3.9 Path Creation Screens 33

A.1 Full page UML Components Diagram. . . . 45
A.2 Full page Deployment View of Best Bike Paths System 46
A.3 SD1. Full page diagram (part 1) 47
A.4 SD1. Full page diagram (part 2) 48
A.5 SD2. Full page diagram 49
A.6 SD3. Full page diagram 50
A.7 SD4. Full page diagram (part 1) 51
A.8 SD4. Full page diagram (part 2) 52
A.9 SD5. Full page diagram 53
A.10 SD6. Full page diagram 54
A.11 SD7. Full page diagram (part 1) 55
A.12 SD7. Full page diagram (part 2) 56
A.13 SD8. Full page diagram 57
A.14 SD9. Full page diagram 58
A.15 SD10. Full page diagram 59

List of Tables

1.1 Acronyms used in this document . . . 2
1.2 Abbreviations used in this document . . . 2
1.3 Revision history of this document . . . 3

2.1 Public API - Authentication . . . 11
2.2 Public API - Trips . . . 11
2.3 Public API - Paths . . . 12
2.4 Public API - Routing . . . 12
2.5 Public API - Users . . . 12
2.6 Internal Inter-Service APIs . . . 13
2.7 External Service Interfaces . . . 13

4.1 Functional Requirements Traceability . . . 36
4.3 Performance Requirements . . . 36
4.5 Software System Attributes . . . 37

6.1 Effort spent by each team member (unit is hours) . . . 42

A.1 Reference table for detailed diagrams in the appendix . . . 44

Chapter 1

Introduction

1.1 Scope

Best Bike Paths (BBP) is a collaborative Platform designed to enhance the cycling experience by facilitating the collection and retrieval of granular road information. It serves a dual purpose: enabling Registered Users to track their performance and actively contribute to the community, while allowing Guest Users to safely navigate and explore the existing map inventory.

To ensure data reliability and safety, the system employs advanced data aggregation logic that resolves conflicting user reports based on freshness and confirmation counts, assigning a definitive quality score to each route. Additionally, BBP minimizes user distraction by automating obstacle detection via sensors, retrieving weather data, and calculating performance statistics, while still offering tools for manual validation and Trip history management.

1.2 Definitions, Acronyms, Abbreviations

1.2.1 Definitions

API Gateway A server that acts as a single entry point for the System, routing requests to the appropriate internal microservices and handling cross-cutting concerns like SSL termination and authentication.

Guest User An unauthenticated User with read-only access to the system features, identified by a temporary session token rather than a permanent credential profile.

Registered User A user who has completed the sign-up process, possessing a persistent profile and write-access to contribute data to the community.

Microservice An architectural style that structures the application as a collection of loosely coupled, independently deployable services organized around business capabilities.

Worker A specialized background component responsible for the asynchronous processing of data. It's structure is similar to the one of a Service but it is not called by any other service in synchronous ways.

Segment The atomic unit of the map graph. Scores and anomalies are mapped onto these specific segments rather than arbitrary coordinates.

Path The spatial representation of a route (geometry), consisting of a sequence of Segments. In this design, a Path can exist independently of a Trip (i.e., manually created).

Trip The temporal recording of a User's activity. It includes metadata such as start time, end time, duration, and weather conditions, and is linked to a specific Path.

Score A dynamic quality metric assigned to road segments. It is calculated by the Aggregation Worker based on the freshness of data and the number of confirmations received from Users.

Thick Client An architectural pattern where the mobile application performs significant data processing (e.g., sensor analysis) and validation locally, ensuring offline capabilities and reducing server load.

1

1.2.2 Acronyms

Acronym	Definition
BBP	Best Bike Paths
DD	Design Document
RASD	Requirements Analysis and Specification Document
REST	REpresentational State Transfer
API	Application Programming Interface
GPS	Global Positioning System
TLS	Transport Layer Security
SSL	Secure Sockets Layer
HTTPS	HyperText Transfer Protocol over Secure Sockets Layer
HTTP	HyperText Transfer Protocol
gRPC	gRPC Remote Procedure Calls
JSON	JavaScript Object Notation
OTP	One Time Password
JWT	JSON Web Token
SMTP	Simple Mail Transfer Protocol
URL	Uniform Resource Location
URI	Uniform Resource Identifier
UI	User Interface
QoS	Quality of Service
DDD	Domain-Driven Development
BFF	Backend-for-Frontend
MitM	Man-in-the-Middle
CQRS	Command Query Responsibility Segregation
DBMS	Database Management System
SQL	Structured Query Language

Table 1.1 – Acronyms used in this document

1.2.3 Abbreviations

Abbreviation	Definition
UC	Use Case
SD	Sequence Diagram

Table 1.2 – Abbreviations used in this document

1.3 Revision History

Revised on	Version	Description
07/01/2026	1.0	Initial release of the document

Table 1.3 – Revision history of this document

1.4 Reference Documents

The information contained in this document is based on the following sources:

[1] E. Di Nitto, M. Rossi, and M. Camilli, *Software Engineering 2 – Requirement Engineering and Design Project*, Politecnico di Milano, A.Y. 2025-2026.

[2] ISO/IEC/IEEE, *Systems and software engineering – Life cycle processes – Requirements engineering*, International Standard ISO/IEC/IEEE 29148:2018.

[3] OpenStreetMap Foundation, *API v0.6 Documentation*[1].

[4] OpenWeather Ltd, *One Call API 3.0 Documentation*[2].

[5] The Kubernetes Authors, *Kubernetes Documentation (v1.35)*[3].

[6] Google LLC, *Flutter Documentation*[4].

1.5 Document Structure

1. **Introduction** outlines the purpose and scope of the design document, providing necessary definitions and acronyms.
2. **Architectural Design** details the system's high-level components, their interactions, the deployment strategy, and the runtime behaviour.
3. **User Interface Design** presents the visual aspect of the system through mock-ups and flow diagrams, defining the user experience.
4. **Requirements Traceability** maps the requirements identified in the RASD to the specific design modules, ensuring full coverage.
5. **Implementation, Integration and Test Plan** defines the testing strategy and the chronological order in which components will be developed and integrated.
6. **Effort spent** the time spent by each group member contributing to the creation of this document.
7. **References** references to documents or tools used in the preparation of this document.

[1] https://wiki.openstreetmap.org/wiki/API_v0.6
[2] https://openweathermap.org/api/one-call-3
[3] https://kubernetes.io/docs/home/
[4] https://docs.flutter.dev/

Chapter 2

Architectural Design

2.1 Overview

The architectural design of the *Best Bike Paths* System is driven by two conflicting non-functional requirements: the need for high-availability in low-connectivity scenarios and the necessity to process massive geospatial datasets for global routing. To resolve this tension, the system adopts a **Distributed Architecture** that combines a **Thick Client** pattern on the User edge with a **Microservices** architecture on the server backend. This architectural vision redistributes the computational load and responsibilities across two main physical nodes:

- **Client Node:** Unlike traditional web-based thin clients, the mobile application acts as a fully functional processing node. It implements the *Thick Client* pattern to ensure the *Offline-First* capability for Trip recording. Instead of merely rendering views, the client hosts a significant portion of the Business Logic and a local Persistence Layer. This allows it to:
  - **Process Data at the Edge:** Raw telemetry from hardware sensors is analysed locally (via the *Motion Analyser*) to extract road quality metrics in real-time. This reduces network bandwidth usage and server ingestion overhead.
  - **Decouple Availability from Connectivity:** Through a local state machine and a mechanism of Store-and-Forward synchronization, the User experience (recording Trips, viewing cached maps) remains seamless regardless of network conditions.
- **Server Node:** The server-side logic adopts a *Microservices Architecture*, where functional domains (Identity, Trips, Routing) are encapsulated in autonomous services. This choice is critical to support independent scaling: compute-intensive tasks like pathfinding can be scaled separately from I/O-bound tasks like User profile management.

The interaction with the client is mediated by an *API Gateway*, which implements the *Backend-for-Frontend (BFF)* pattern. It provides a unified entry point, handling security offloading (SSL), routing, and protocol translation, shielding the internal complexity of the cluster from the mobile app.

The system employs a polyglot persistence strategy optimized for geospatial performance. While most microservices strictly adhere to the *Database-per-Service* pattern to ensure decoupling, the Routing domain introduces a strategic deviation. To handle complex graph traversals efficiently, the architecture implements a variation of the *CQRS (Command Query Responsibility Segregation)* pattern:

- The *Aggregation Worker* (Write Model) asynchronously processes massive updates to road segment reliability scores.
- The *Routes Manager* (Read Model) accesses the same shared geospatial dataset to compute optimal Paths with low latency.

This hybrid approach ensures that the rigorous isolation required for User data coexists with the high performance shared state requirements of the routing engine.

2.2 Component View

In this section, the main components of the BBP System are outlined, with an accurate description of what each of them aims to accomplish. Every component is approximately derived from one of the product functions discussed in Section 2.2 of the RASD. Further, components are grouped together by affinity in high-level *macro-categories* to form microservices, obtaining what is usually referred to as *bounded contexts* in Domain-Driven Development

(DDD). Components also expose interfaces with methods to be invoked from external entities and communicate with some databases.

2.2.1 Components Diagrams

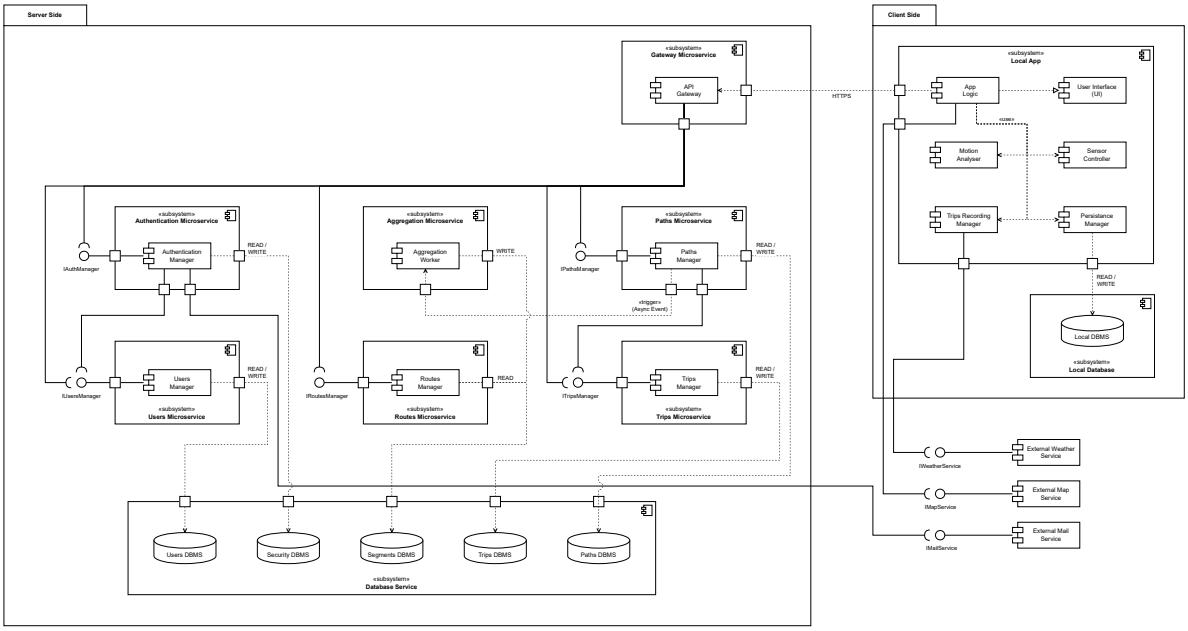


Figure 2.1 – UML Components Diagram.

Please refer to Figure A.1 (Page 45) for a bigger version.

2.2.2 Components Descriptions

The System architecture is divided into two main processing environments: the *Client Side*, responsible for real-time data collection and local interactions, and the *Server Side*, which handles data consolidation, synchronization, and global analytics.

Client Side Components

The client application acts as a *Thick Client*, designed to operate autonomously even in the absence of network connectivity (e.g., when recording a Trip in remote locations).

- **Sensor Controller:** This component serves as the hardware abstraction layer. It interfaces directly with the device's physical sensors (GPS and Accelerometer) to retrieve raw telemetry data streams at high frequency.
- **Motion Analyzer:** A critical module for edge computing. It processes the raw data stream from the Sensor Controller locally to detect specific events (e.g., sudden braking, potholes, or surface irregularities) and compute real-time road quality metrics. This reduces the bandwidth usage by transmitting only processed insights rather than raw sensor noise.
- **Trips Recording Manager:** Orchestrates the active Trip session. It subscribes to the processed data from the Motion Analyzer, organizes it into a coherent Trip structure (associating timestamps and locations), and manages the start/stop/pause logic of a recording.
- **App Logic:** The central coordinator of the mobile application. It manages the application state, handles navigation between UI screens, and orchestrates communication with the backend.

- **Persistence Manager:** This component implements the *Offline-First* pattern. It abstracts the underlying *Local DBMS*, handling all read/write operations on the device. It ensures that Trip data is securely stored locally immediately after recording and manages the synchronization queue when the device regains connectivity. It also stores authentication data, such as JWTs.
- **Local DBMS:** A lightweight relational database (e.g., SQLite) residing on the User's device, used for caching Paths and storing unsynchronized Trips.

Server Side Components

The backend is structured as a generic *Microservices Architecture*, where functional domains are encapsulated in independent subsystems to ensure scalability and fault isolation.

- **Gateway Microservice:** It acts as the single entry point for all client requests. It handles SSL termination, request routing, and load balancing, forwarding calls to the appropriate microservice interface (e.g., `IAuthManager`, `ITripsManager`).
- **Authentication & User Microservices:** These subsystems handle the identity management (via the `Authentication Manager`) and User profile data (via the `Users Manager`), managing credentials and personal settings stored in their respective databases.
- **Trips Microservice:** Manages the lifecycle of Trip entities (temporal metadata). As depicted in the diagram, it has a functional dependency («`use`») on the *Paths Microservice*: when a Trip is saved, the Trips Manager invokes the Paths Manager to persist the associated geospatial data.
- **Paths Microservice:** Responsible for the catalogue of Paths (CRUD operations). Crucially, upon modifying a Path's visibility, it triggers an asynchronous event («`trigger`») towards the *Aggregation Worker* to initiate the map update process without blocking the User response.
- **Routes Microservice:** Dedicated to complex geospatial operations. It handles pathfinding queries and spatial searches, interacting with the geospatial data in the *Segments DBMS*.
- **Aggregation Microservice:** A background processing unit responsible for maintaining the global map consistency. It does not expose any interfaces as it listens for change events. When triggered, it recalculates the scores of road segments by merging new User data with historical records.
- **Database Service:** The persistence layer groups the System's databases. While most microservices maintain exclusive ownership of their data (e.g., *Trips DBMS*, *Users DBMS*) to foster decoupling, the *Segments DBMS* represents a notable exception. It acts as a Shared Data Store accessed by multiple services (`Routes Manager` and `Aggregation Worker`). This design choice is driven by the specific requirement to perform heavy geospatial calculations (Routing) and global updates (Reliability Scoring) on the same dataset without the overhead of data duplication or continuous synchronization.

External Services

The System relies on third-party providers to fulfill specific capabilities that are outside the core domain logic. These services are integrated via abstract interfaces to ensure modularity and avoid vendor lock-in.

- **External Map Service:** Provides the fundamental geospatial infrastructure for the frontend. The Client Application consumes this service to render map tiles, perform geocoding operations, and support the User Interface during track visualization. The System is designed to support open solutions like **OpenStreetMap**[1] and **OpenCycleMap**[2], as well as proprietary providers like **Google Maps Platform**, depending on the required licensing and tile resolution.
- **External Weather Service:** A third-party API used to enrich Trip data context. The Client Side interacts with this service to fetch environmental conditions (e.g., temperature, weather status) corresponding to the

[1] https://www.openstreetmap.org/
[2] https://www.opencyclemap.org/

specific location and time of a recorded Trip. A standard provider for this capability is **OpenWeatherMap**[3], but the architecture supports any REST-compliant weather API (e.g., AccuWeather, WeatherAPI).

- **External Mail Service:** An infrastructure provider responsible for the delivery of transactional emails. It is invoked by the **Authentication Manager** to dispatch One-Time Passwords (OTP) during the User registration or password reset flows. Industry-standard SMTP providers such as **SendGrid**[4] or **Mailgun**[5] are suitable candidates for high-deliverability requirements.

2.3 Deployment View

2.3.1 Overview

The Deployment View illustrates the mapping of software artifacts onto the physical hardware nodes. The *Best Bike Paths* System adopts a distributed architecture to ensure scalability, fault tolerance, and offline capabilities. The physical topology is organized into a **3-Tier** structure (Client, Server, Data), which hosts the **6 Logical Layers** defined in the architectural design.

The deployment strategy relies heavily on containerization (Docker) and orchestration (Kubernetes) for the backend, while the client side is designed as a *Thick Client* to perform local data processing and persistence.

2.3.2 Deployment Diagram

Figure 2.2 depicts the UML Deployment Diagram of the System. It highlights the nesting of execution environments, showing how the application logic is encapsulated within the mobile operating System on the client side, and within the Kubernetes cluster on the server side.[6]

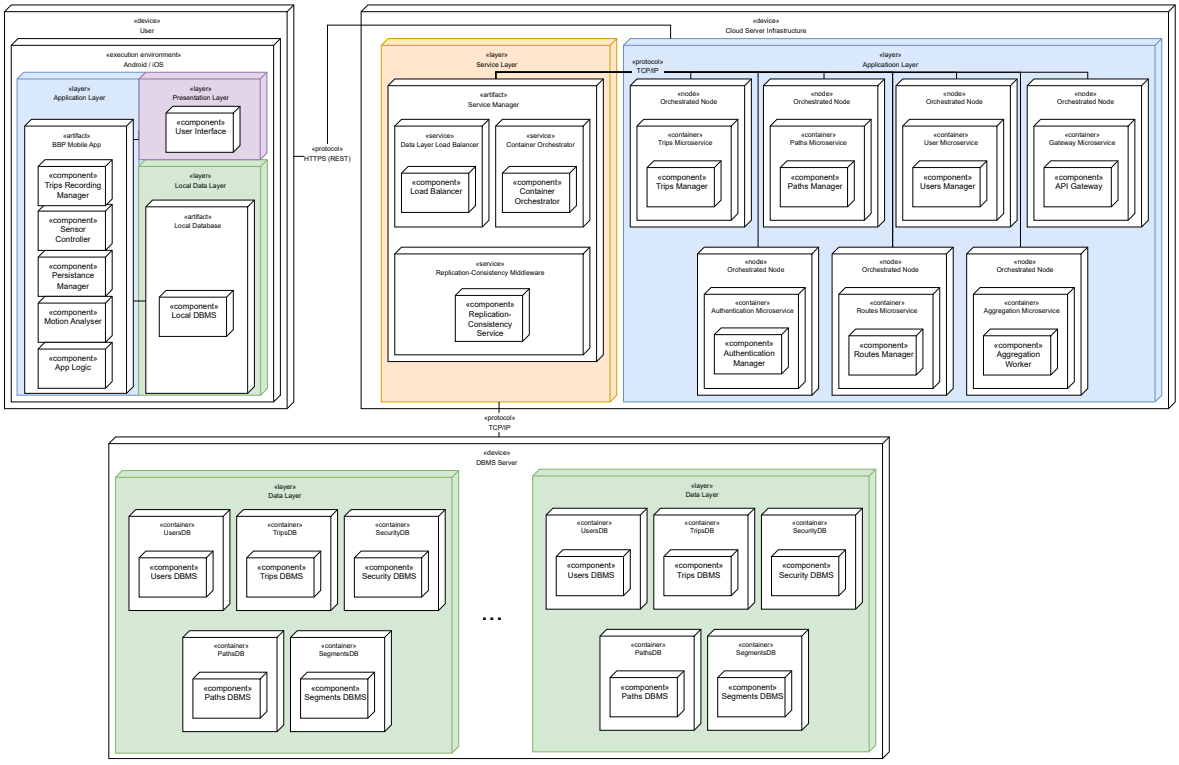


Figure 2.2 – UML Deployment Diagram.

[3] https://openweathermap.org/
[4] https://sendgrid.com/
[5] https://www.mailgun.com/
[6] Inter-node connections are not shown to avoid readability issues, but please note that nodes can communicate via HTTP/gRPC protocol.

Please refer to Figure A.2 (Page 46) for a bigger version.

2.3.3 Node mapping

The following sections describe the specific configuration of each node and the software artifacts deployed on them.

Tier 1: User Client Device

This node represents the end-User's smartphone. Given the *Thick Client* nature of the application, this node bears a significant computational load.

- **Hardware Node**: Smartphone (Android / iOS).
- **Execution Environment**: Mobile Operating System (Android OS or iOS).
- **Hosted Layers**:

1. **Presentation Layer**: Hosted within the Mobile App artifact, responsible for UI rendering.
2. **Mobile Application Layer**: Hosted within the Mobile App artifact. It includes the *Motion Analyzer* component for real-time sensor processing.
3. **Client Data Layer**: Hosted within the *Local Database* artifact. It ensures data persistence when the device is offline. A possible implementation might be through SQLite.

Tier 2: Backend Server Infrastructure

This node represents the cloud infrastructure. It is abstracted by the orchestration layer, allowing the System to treat a cluster of physical machines as a single execution environment.

- **Hardware Node**: Cloud Virtual Machines (Worker Nodes).
- **Execution Environment**: **Kubernetes Cluster** (Orchestration Layer).
- **Hosted Layers**:

4. **Server Application Layer**: Composed of Docker Containers (Pods) running the microservices (*User Service*, *Trip Service*, *Path Service*, *Routes Service*, *Aggregation Worker*) and the API Gateway.
5. **Service Orchestration Layer**: The Kubernetes control plane itself, managing the lifecycle and networking of the artifacts mentioned above.

Tier 3: Data Persistence Server

To ensure data integrity and isolation, the primary database runs on a dedicated environment, separated from the application logic.

- **Hardware Node**: Dedicated Database Server or Managed Cloud Database Instance.
- **Execution Environment**: PostgreSQL Engine.
- **Hosted Layers**:

6. **Server Data Layer**: Hosted within the *Spatial DBMS* artifact (PostgreSQL + PostGIS extension). It stores the persistent state of the System (Users, Paths, Trips, Segments).

2.3.4 Detailed Layer Decisions

Client Logic and Local Persistence

The decision to implement a **Thick Client** architecture with a dedicated **Client Data Layer** (SQLite) is driven by the "Offline-First" requirement. Instead of relying on a continuous network stream, the mobile application acts as a stand-alone unit during the Trip recording.

- **Local Pre-processing**: The *Mobile Application Layer* processes high-frequency sensor data locally. Only aggregated events (e.g., detected potholes) and simplified Path coordinates are stored, significantly reducing bandwidth consumption.
- **Synchronization**: The local database acts as a temporary buffer. Data synchronization is deferred until a Trip ends and a stable connection is available.

API Gateway and Load Balancing

To decouple the client from the internal microservices, an **API Gateway** has been introduced in the Server Application Layer.

- **Unified Interface**: It exposes a stable, public API to the mobile clients, hiding the complexity of the internal microservices network.
- **Scalability**: The API Gateway is stateless. This allows the orchestration layer to **replicate** the Gateway across multiple pods. A load balancer distributes incoming traffic among these replicas, preventing the Gateway itself from becoming a bottleneck or a single point of failure.

Service Orchestration, Resiliency and Scalability

The **Service Orchestration Layer** (Kubernetes) is the core enabler of the System's reliability. It manages the lifecycle of the Server Application Layer components through:

- **Replication**: Critical services (such as the *User Service* and *Trip Service*) are deployed with a replication factor greater than one ($N \geq 2$). If one instance is busy or fails, traffic is seamlessly routed to the others.
- **Resiliency (Self-Healing)**: The orchestrator continuously monitors the health of the pods. If a microservice crashes (e.g., due to a runtime error), Kubernetes automatically restarts the container to restore the desired state without human intervention.
- **Horizontal Scalability**: The layer monitors resource usage (CPU/RAM). During peak traffic, it automatically provisions additional replicas of the stressed services (Autoscaling), ensuring consistent performance.

Server Data Layer Optimization

For the **Server Data Layer**, the choice of a Spatial DBMS (PostgreSQL with PostGIS) is dictated by the domain efficiency. Storing geospatial data as native geometric objects (Point, Linestring) allows the System to perform complex spatial queries (e.g., finding potholes within a radius) using optimized R-Tree indices, which is significantly faster than performing mathematical calculations at the application level.

2.4 Component Interfaces

2.4.1 REST API Interface

The interaction between the Client Tier and the Server Tier, as well as the communication among internal microservices, relies on a RESTful API architecture. This section defines the structural patterns, addressing schemes,

and the logical division between public and internal interfaces.

General Design Principles

All APIs share the following technical characteristics to ensure consistency and interoperability:

- **Protocol**: HTTPS (TLS 1.3) for public communication; HTTP/gRPC for internal cluster traffic.
- **Data Format**: JSON (JavaScript Object Notation) is used for all request bodies and response payloads.
- **Date Format**: ISO 8601 Strings (e.g., `2025-01-07T14:30:00Z`).
- **Error Handling**: Standard HTTP Status Codes (2xx for success, 3xx for redirects, 4xx for client errors, 5xx for server errors) accompanied by a JSON error object.

URI Structure and Namespaces

The System distinguishes between APIs exposed to the outside world and APIs used strictly for inter-service communication.

Public Interface Strategy The public API is exposed via the **API Gateway**. It acts as the single entry point for the mobile application.

- **Base Domain**: `https://api.bestbikePaths.com`
- **Versioning**: All public URIs are versioned to ensure backward compatibility.
- **Pattern**: `METHOD /api/v1/<resource>/...`

Internal Interface Strategy Internal APIs are accessible only within the Kubernetes cluster private network. They are not reachable from the public internet.

- **Purpose**: These endpoints allow microservices to exchange data or trigger actions (e.g., Auth Service telling User Service to create a profile) without exposing these operations to the client.
- **Pattern**: `METHOD /internal/<service-name>/<action>`

API Macro Areas

The API resources are organized into logical macro-areas corresponding to the underlying microservices domain[7]:

- `/auth`: Manages registration, login, OTP verification, and token refresh.
- `/users`: Handles User profiles, settings, and personal statistics.
- `/trips`: Manages Trip recording, upload, and history retrieval.
- `/paths`: Handles the retrieval of map segments, tiles, and pothole data.
- `/routes`: Manages Pathfinding operations, calculating optimal routes based on distance and road safety scores.

2.4.2 Public API Endpoints (Gateway)

The following tables detail the REST interface exposed by the API Gateway.[8]

All URIs listed below are relative to the base path: **`/api/v1`**.

[7] Note that the *Aggregation Worker* does not expose any API as it works by listening to events.

[8] Note on Status Codes: Standard technical errors (e.g., *500 Internal Server Error*) and generic authorization errors (e.g., *401 Unauthorized*) are omitted from the tables. All endpoints assume a valid JWT token in the `Authorization` header unless stated otherwise.

Authentication (Prefix: /auth)

Method	URI	Request Payload / Query	Response
POST	/signup	**Payload:** `{ "email": "...", "password": "...", "userData": {...} }` **Query:** *None*	**200 OK**: OTP sent to email. **409 Conflict**: Email already registered.
POST	/verify-otp	**Payload:** `{ "email": "...", "code": "..." }` **Query:** *None*	**201 Created**: `{ "token": "JWT...", "userId": "..." }` **400 Bad Request**: Invalid code.
POST	/login	**Payload:** `{ "email": "...", "password": "..." }` **Query:** *None*	**200 OK**: `{ "token": "JWT...", "userId": "..." }` **401 Unauthorized**: Invalid credentials.
POST	/guest-token	**Payload:** `{ "deviceId": "uuid..." }` **Query:** *None*	**200 OK**: `{ "token": "GuestJWT...", "expiresIn": "..." }`

Table 2.1 – Public API - Authentication

Trips (Prefix: /trips)

Method	URI	Request Payload / Query	Response
POST	/save	**Payload:** `{ "tripData": {...}, "geometry": {...}, "visibility": "PUBLIC"|"PRIVATE" }` **Query:** *None*	**201 Created**: `{ "tripId": "..." }` **400 Bad Request**: Invalid format.
DELETE	/{id}	**Payload:** *None* **Query:** `?cascade=true|false`	**204 No Content**: Deleted. **403 Forbidden**: Not owner. **404 Not Found**: Trip ID invalid.
GET	/list	**Payload:** *None* **Query:** `?page=...&limit=...`	**200 OK**: `{ "trips": [ { ... } ], "total": "..." }`
GET	/{id}	**Payload:** *None* **Query:** *None*	**200 OK**: `{ "tripData": {...}, "geometry": {...}}` **404 Not Found**: Invalid ID. **403 Forbidden**: Not owner.

Table 2.2 – Public API - Trips

2

Paths (Prefix: /paths)

Method	URI	Request Payload / Query	Response
POST	/save	**Payload:** `{ "geometry": {...}, "visibility": "PUBLIC"|"PRIVATE" }` **Query:** *None*	**201 Created**: `{ "pathId": "..." }` **400 Bad Request**: Invalid geometry format.
DELETE	/{id}	**Payload:** *None* **Query:** *None*	**204 No Content**: Deleted. **409 Conflict**: Linked to a Trip. **404 Not Found**: Invalid ID. **403 Forbidden**: Not owner.
POST	/{id}/chg-vis	**Payload:** `{ "visibility": "PUBLIC"|"PRIVATE" }` **Query:** *None*	**200 OK**: Status updated. **403 Forbidden**: Not owner. **404 Not Found**: Path ID invalid.
GET	/list	**Payload:** *None* **Query:** `?page=...&limit=...`	**200 OK**: List of User's Paths.
GET	/{id}	**Payload:** *None* **Query:** *None*	**200 OK**: `{ "geometry": {...}, "visibility": "PUBLIC"|"PRIVATE" }` **404 Not Found**: Invalid ID. **403 Forbidden**: Not owner.

Table 2.3 – Public API - Paths

Routes (Prefix: /routes)

Method	URI	Request Payload / Query	Response
GET	/calculate	**Payload:** *None* **Query:** `?start=lat,lon & end=lat,lon`	**200 OK**: `[ { "routeId": "...", "segments": [...], "score": "..." } ]` **404 Not Found**: No route found.

Table 2.4 – Public API - Routing

Users (Prefix: /users)

Method	URI	Request Payload / Query	Response
GET	/{id}/profile	**Payload:** *None* **Query:** *None*	**200 OK**: `{ "userData": {...}, "stats": {...} }` **404 Not Found**: User ID not found. **403 Forbidden**: Mismatch owner/caller.

Table 2.5 – Public API - Users

Internal Microservices Interfaces

Most internal endpoints managed by the API Gateway mirror the Public API definitions with a 1:1 mapping, simply forwarded to the respective microservice within the cluster (e.g., `/internal/auth/login` accepts the same payload as the public counterpart).

However, specific endpoints exist solely for **Inter-Service Communication** to support business transactions that span multiple domains. These are detailed below.

For the sake of clarity and uniformity with the Public API documentation, the interfaces below are described using logical REST-like resource paths. However, in the actual deployment, these high-frequency internal interactions are implemented using **gRPC** (over HTTP/2) and **Protocol Buffers** to minimize latency and serialization overhead.

Target Service	Method & URI	Caller	Payload / Response
Users Manager	`POST /create`	**Auth Service**	**In:** `{ "userId": "...", "email": "...", "name": "..." }` **Out:** 201 Created
Paths Manager	`GET /{id}/geometry`	**Aggregation Worker**	**In:** *None (GET)* **Out:** 200 OK `{ "geometry": {...}, "metadata": {...} }`
Paths Manager	`DELETE /{id}`	**Trips Service**	**In:** *None* **Out:** 204 No Content
Paths Manager	`PATCH /{id}`	**Trips Service**	**In:** `{ "tripId": null }` **Out:** 200 OK

Table 2.6 – Internal Inter-Service APIs

2.4.3 External Service Interfaces

The System relies on third-party services to fulfil specific capabilities. The following table summarizes the external contracts consumed by both the Mobile Client and the Backend Microservices.

To maintain flexibility and avoid *vendor lock-in*, the interfaces described below are defined as *logical abstractions*. Their use in the sequence diagrams are an example of a possible implementation. The *Adapter Design Pattern* can be used to eventually translate calls.

External Service	Consumer	Purpose	Data Exchanged
Map Service	**Client App**	Rendering map tiles and Geocoding addresses.	**Out:** Address string or Tile Coordinates (x,y,z). **In:** Lat/Lon coordinates or PNG Tile images.
Email Provider	**Auth Service**	Sending OTP codes for 2FA and account verification.	**Out:** Recipient Email, OTP Code, Template ID. **In:** Delivery Status.
Weather Service	**Trip Manager**	Enriching Trip data with weather conditions.	**Out:** Lat/Lon, Timestamp. **In:** Temperature, Condition (Rain/Sun), Humidity.

Table 2.7 – External Service Interfaces

2.5 Runtime View

What follow are the ten Sequence Diagrams corresponding to the ten Use Cases illustrated in the RASD. A bigger version of the diagrams is available in the Appendix to this document.

2

[SD1] Sign Up by a User

The sequence diagram illustrates the registration process for an Unregistered User, enforcing a secure two-step verification flow. The interaction begins with the User inputting their credentials via the *User Interface*. The *App Logic* first performs local validation on the input fields; upon success, it transmits the request through the *API Gateway* to the *Authentication Service*. This service queries the *SecurityDBMS* to verify the uniqueness of the provided email. If the email is available, a new record with a `PENDING` status is created, and the *EmailProvider* is triggered to dispatch a One-Time Password (OTP) to the User.

In the second phase, the User submits the received OTP for verification. The *Authentication Service* validates the code against the *SecurityDBMS*. Upon successful validation, the account status is updated to `ACTIVE`. To ensure separation of concerns, the *Authentication Service* subsequently invokes the *Users Manager* via an internal API to initialize the User's persistent profile in the *UsersDBMS*. Finally, a JWT is generated and returned to the *App Logic*, which locally persists the session token and redirects the User to the application dashboard.

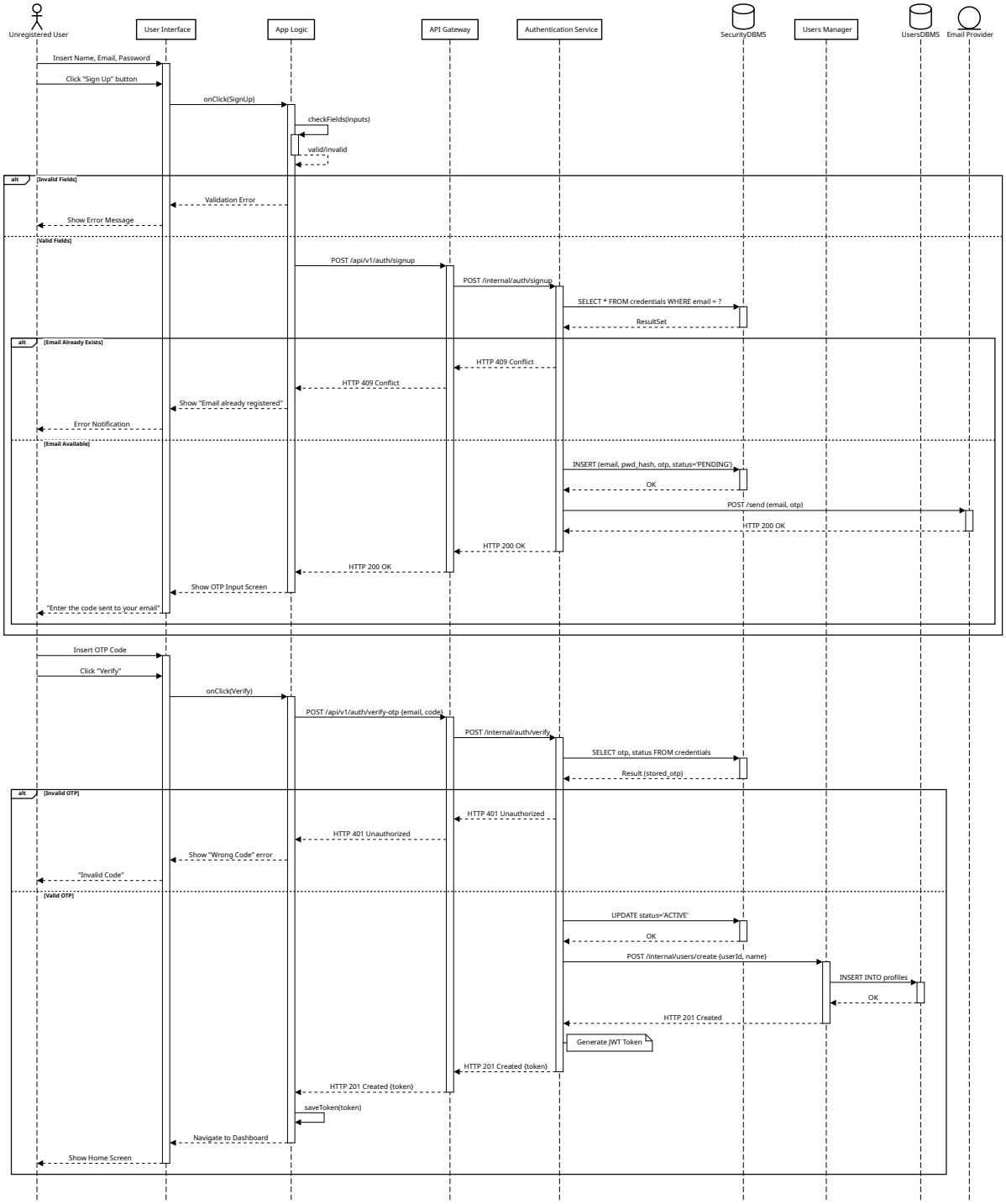


Figure 2.3 – SD1. Sign Up by a User

[SD2] Log In by a User

The diagram details the authentication procedure for a Registered User, which involves both credential verification and the initialization of the local application context. The process initiates with the User submitting their email and password via the *User Interface*. Following a local format validation by the *App Logic*, the credentials are transmitted through the *API Gateway* to the *Authentication Service*. This component retrieves the stored password hash and account status from the *SecurityDBMS* to perform a cryptographic verification. Upon success, the service generates a JWT pair, which is returned to the client and persisted in the device's secure storage.

Subsequently, to satisfy the System's Thick Client architectural requirements, the *App Logic* automatically triggers a data synchronization routine. It requests the User's full profile details using the newly acquired access token. The *API Gateway* routes this request to the *Users Manager*, which queries the *UsersDBMS* for personal attributes, preferences, and statistics. This payload is downloaded and cached in the local SQLite database, ensuring the application is immediately functional, even in potential offline scenarios, before finally redirecting the User to the main dashboard.

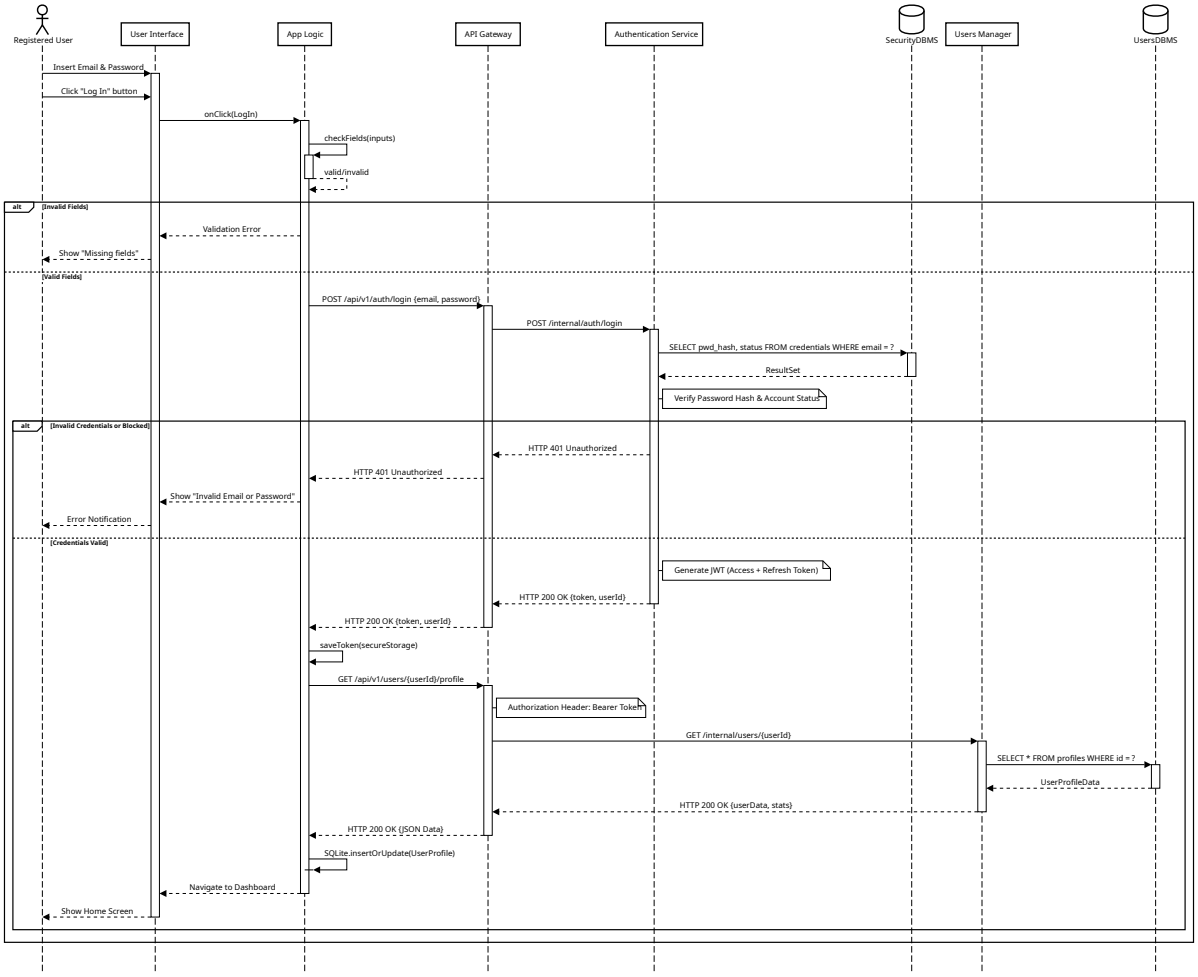


Figure 2.4 – SD2. Log In by a User

[SD3] Access as Guest

The sequence outlines the procedure for granting limited access to an Unregistered User via an anonymous session. The process is triggered by the User selecting the Guest option on the *User Interface*. The *App Logic* initiates a request to the *API Gateway*, which forwards the call to the *Authentication Service*. Unlike standard authentication, this service does not query the User database; instead, it generates an ephemeral JWT with restricted privileges (Guest Role), valid for a limited duration, and returns it to the client.

Upon receiving the token, the *App Logic* securely stores it for subsequent read-only API calls. Crucially, to maintain architectural consistency with the Registered User flow, the System instantiates a placeholder profile within the local SQLite database. This allows the application to maintain a persistent link at further openings.

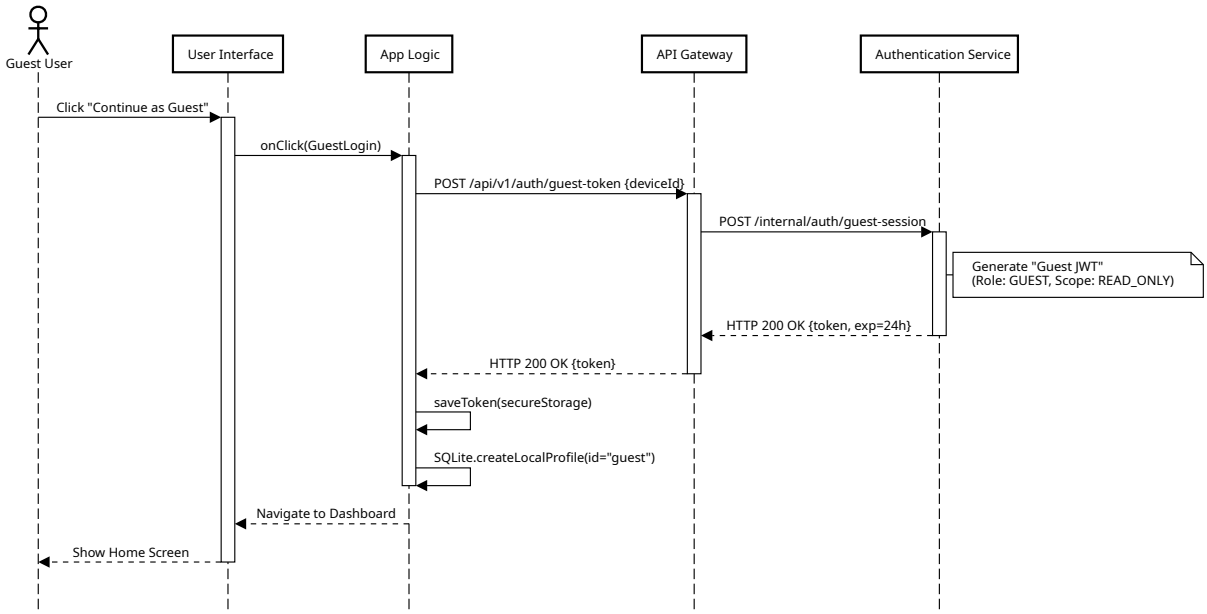


Figure 2.5 – SD3. Access as Guest

[SD4] Record a Trip

The diagram demonstrates the core offline capability of the System, detailing how a Trip is recorded using strictly local resources while attempting to enrich data when connectivity permits. The process begins with the Registered User initiating the recording via the *User Interface*. The *Trip Recording Manager* instantiates a new session object and commands the *Sensor Controller* to activate the GPS, accelerometer, and gyroscope hardware at high frequency to ensure granular data capture.

During the active recording phase, the System operates in a continuous sampling loop. Incoming raw sensor data is routed to the *Motion Analyser*, which performs edge computing tasks such as noise filtering and motion state detection. If the User is stationary, the System triggers the *Smart Pause* logic to suspend the timer. Conversely, when motion is detected, the *Trip Recording Manager* updates the Trip statistics, checks for potential anomalies (e.g., potholes), and buffers the data points in memory.

Upon termination, the System disables the sensors to conserve battery. Before final persistence, the logic attempts to retrieve environmental context by querying an external *Weather Service* using the last known GPS coordinates. This interaction is designed to be non-blocking: if the API is reachable, metadata are retrieved; otherwise, in case of timeout or connectivity errors or unavailability, the weather data is simply omitted. Finally, the *Trip Recording Manager* flushes the buffered data, including the Path, statistics, anomalies, and any optional weather information, to the *Local SQLite DB*, ensuring persistence before displaying the summary to the User.

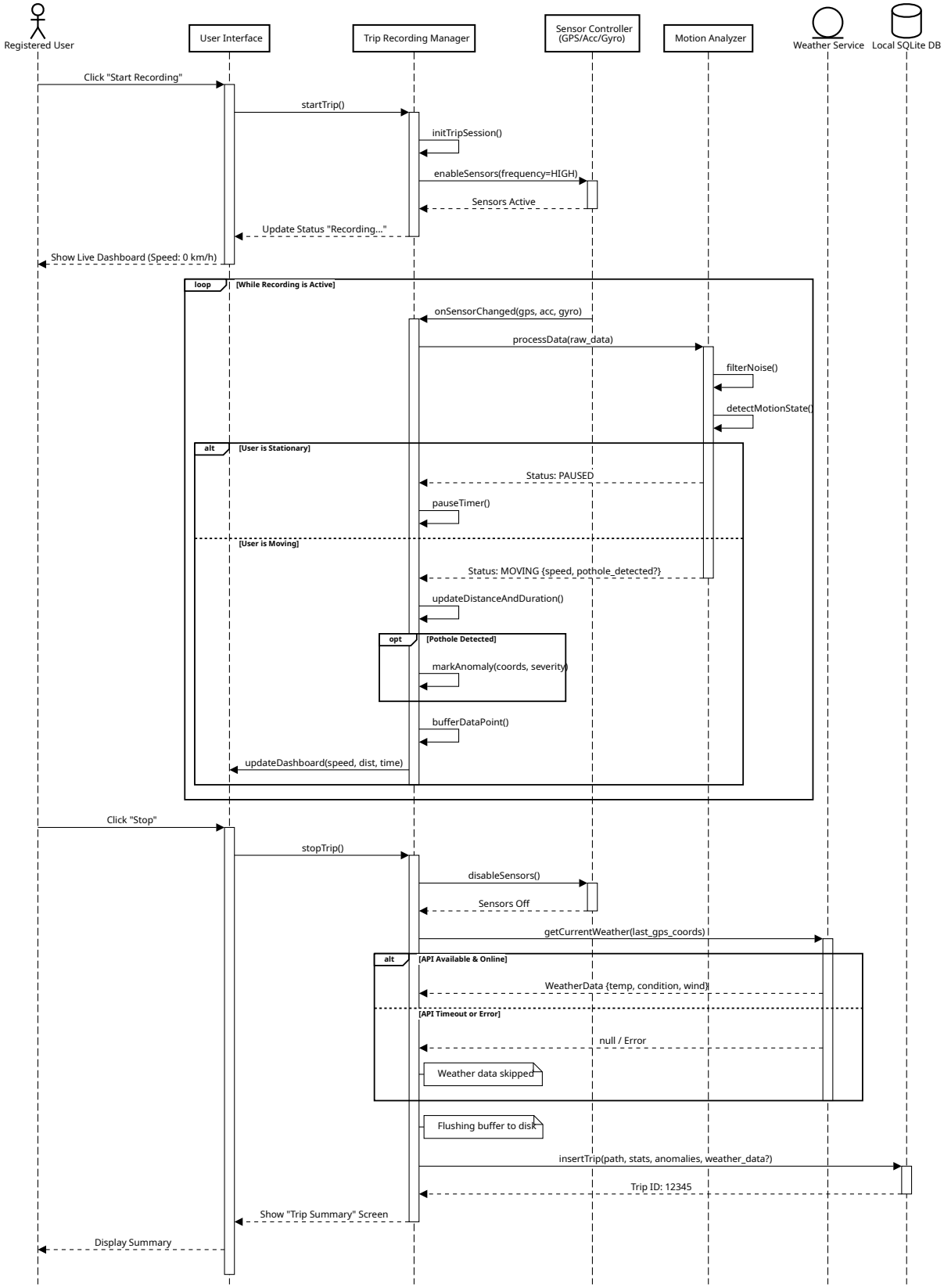


Figure 2.6 – SD4. Record a Trip

2

[SD5] Validate a Trip and a Path

This sequence diagram depicts the final validation and persistence process of a *Trip* and its associated *Path* at the end of a recording session. Consistent with the System's *Thick Client* architecture and *Offline-First* approach, the initial validation phase where the User confirms obstacles and defines the Path status is performed locally within the *App Logic* without interacting with the backend.

Once the User confirms the save operation, the *App Logic* transmits the complete payload (including Trip metadata, Path geometry, and validated obstacles) via the *API Gateway* to the *Trips Manager*. This service acts as the orchestrator for the transaction: it first persists the temporal metadata (e.g., duration, average speed) into the *Trips DBMS*, and then synchronously invokes the *Paths Manager* to handle the geospatial data storage in the *Paths DBMS*.

If the User sets the Path visibility to *Public*, the process triggers an asynchronous consistency mechanism. The *Paths Manager* notifies the *Aggregation Worker*, which runs in the background to retrieve historical data, merge it with the new contribution, and update the *Score* of the affected road segments in the global map. This ensures that the User receives immediate confirmation of the saved Trip, while the computationally expensive aggregation task is handled separately.

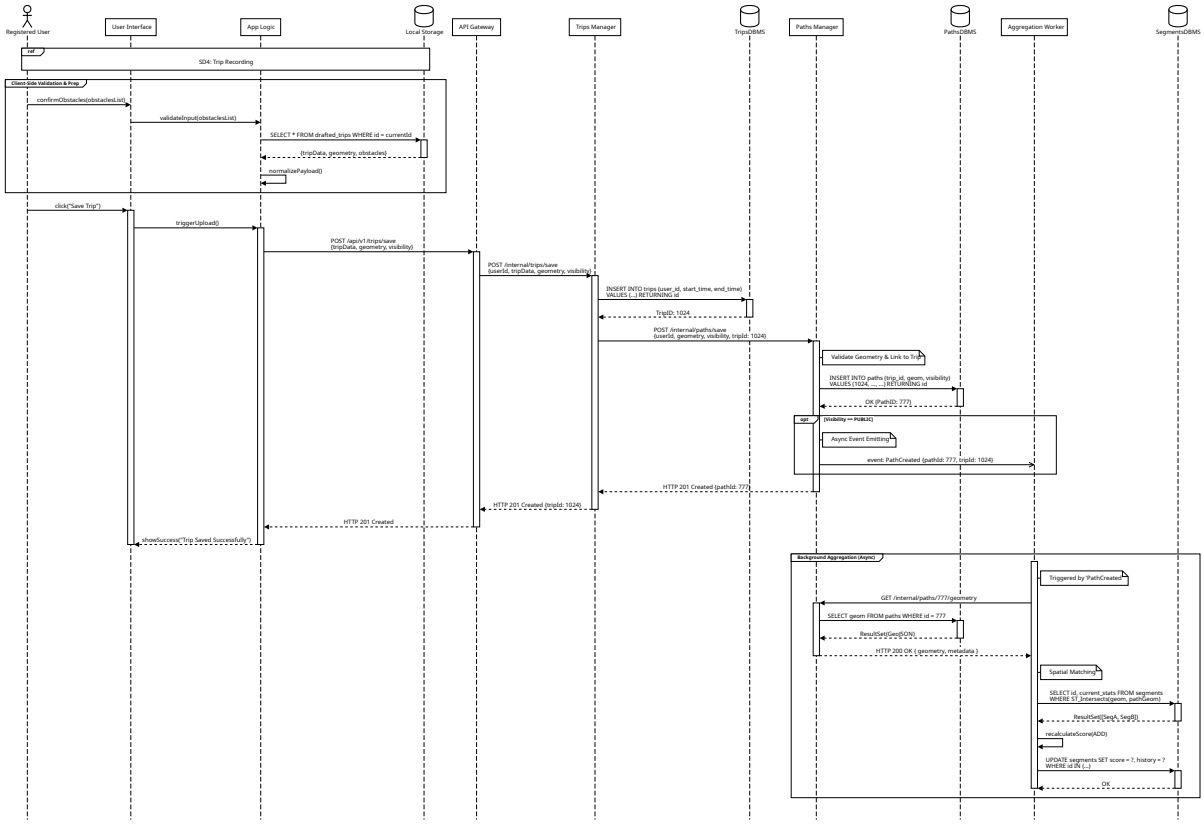


Figure 2.7 – SD5. Validate a Trip and a Path

[SD6] Change Path's visibility

This sequence diagram illustrates the process of modifying the visibility status of an existing Path (e.g., switching from *Private* to *Public* or vice versa).

When the User toggles the visibility setting via the *User Interface*, the *App Logic* forwards the request to the *Paths Manager* through the *API Gateway*. The manager synchronously updates the Path's metadata in the *Paths DBMS* to reflect the new state.

Crucially, this operation triggers a background consistency mechanism to ensure the global map remains accurate. Immediately after the local update, the *Paths Manager* sends an asynchronous notification to the *Aggregation Worker*. Depending on the new visibility state, the worker adjusts the global statistics: if set to *Public*, the worker integrates the Path's segments statuses in the corresponding public road segments by the algorithm, valuing freshness and confirming reports; conversely, if set to *Private*, it removes the Path's contribution and computes a new status based on history. This asynchronous design pattern allows the System to provide immediate feedback to the User (*Visibility Updated*) while the computationally intensive map re-aggregation is performed by the worker in the background.

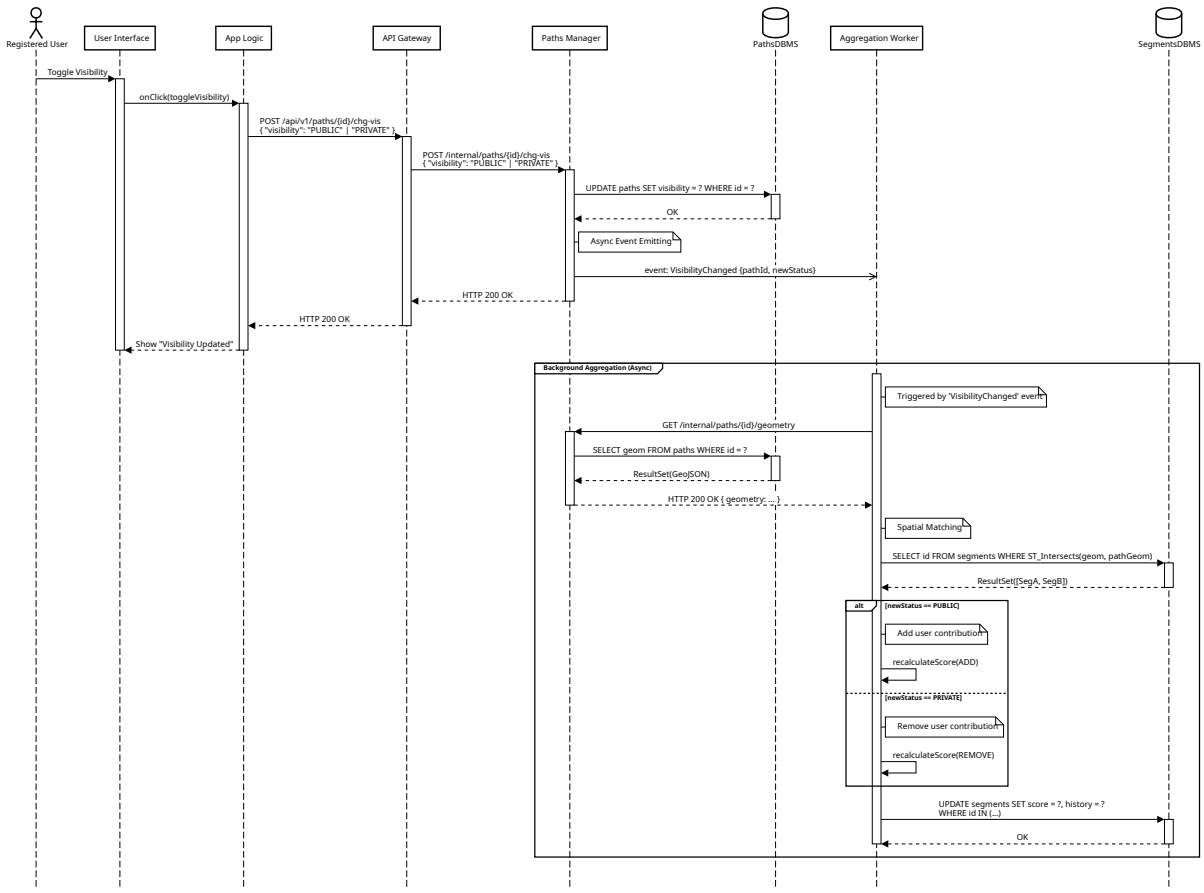


Figure 2.8 – SD6. Change Path's visibility

[SD7] Create a Path manually

The diagram depicts the flow for manually defining a new route without recording a live Trip. The process initiates with the Registered User interacting with the *User Interface*, which first retrieves map tiles and road geometry from the *Map Service* (e.g., via snapping APIs) to render the interface. The User iteratively defines the track geometry and attributes via the *App Logic*. Upon confirmation, the *App Logic* transmits the data through the *API Gateway* to the *Paths Manager*.

This component validates the geospatial consistency of the segments; if the data is valid, it persists the new Path in the *Paths DBMS*. In accordance with the System's data quality requirements, if the Path's visibility is set to *Public*, the *Paths Manager* emits an event to the *Aggregation Worker*. This worker retrieves the geometry via the internal API, calculates the scores of the affected road segments, and updates the *Segments DBMS* to reflect the User's contribution on the global map.

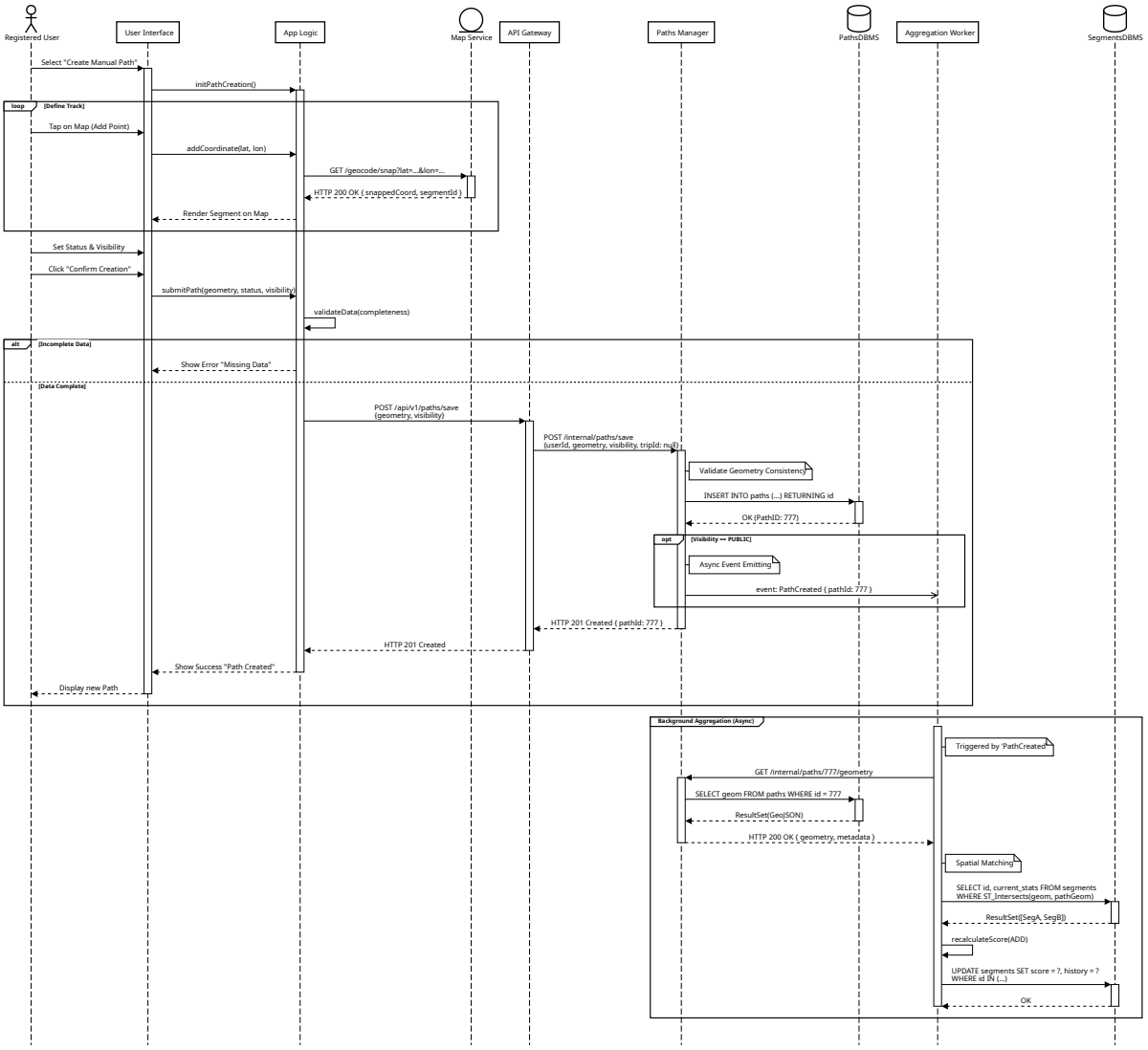


Figure 2.9 – SD7. Create a Path Manually

[SD8] Search for a Route

This sequence diagram outlines the workflow when a User searches for suitable Paths based on specific criteria (e.g., location, status, or length).

The process begins with the User submitting the search parameters via the *User Interface*. The *App Logic* forwards the query through the *API Gateway* to the *Routes Manager*. This component is responsible for handling geospatial queries: it accesses the *Segments DBMS* to identify routes that match the User's requirements.

Once the matching items are retrieved, the *Routes Manager* aggregates the results and returns a list of candidate routes to the *App Logic*, which displays them to the User.

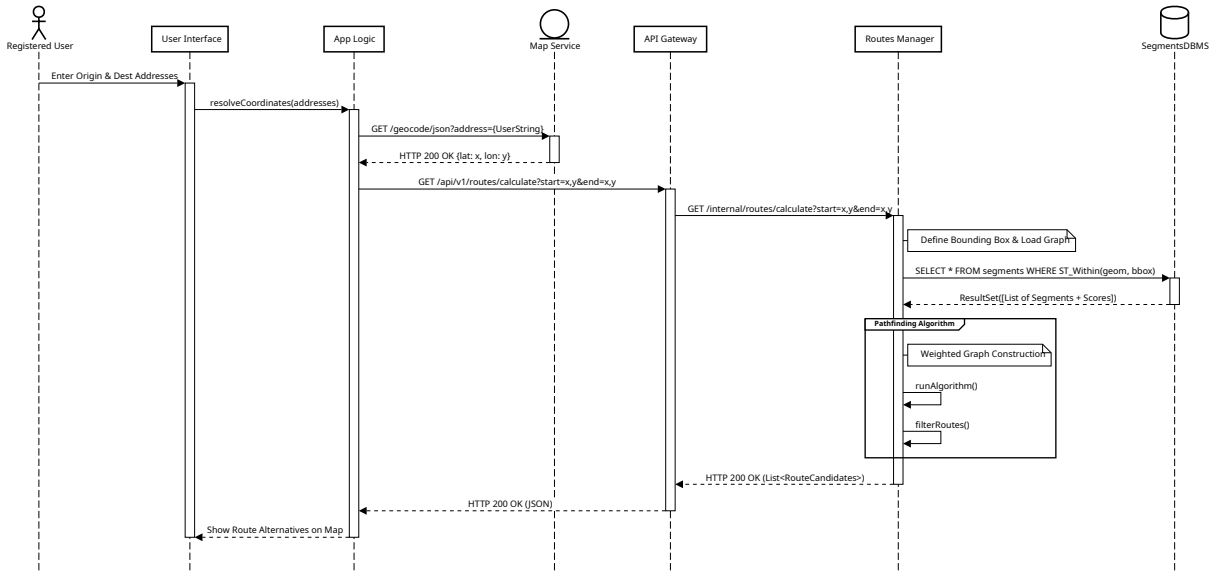


Figure 2.10 – SD8. Search for a Route

[SD9] Delete a Trip

The diagram illustrates the interaction flow when a Registered User initiates the deletion of a recorded Trip. The process begins at the *User Interface* layer, which prompts the User to specify the scope of the deletion: removing only the Trip statistics (keeping the Path as a manual entry) or deleting both the Trip and its associated geographical Path. The *App Logic* forwards the request to the *API Gateway*, which orchestrates the necessary calls to the *Trips Manager*.

In both scenarios, the *Trips Manager* performs a hard delete of the temporal metadata from the *Trips DBMS*. If the User opts to delete both entities, the *Trips Manager* invokes the *Paths Manager* via the internal API. Crucially, if the Path was *Public*, the *Paths Manager* triggers the *Aggregation Worker* (passing the geometry in the deletion event) to remove the User's contribution from the global road segment scores, ensuring data consistency.

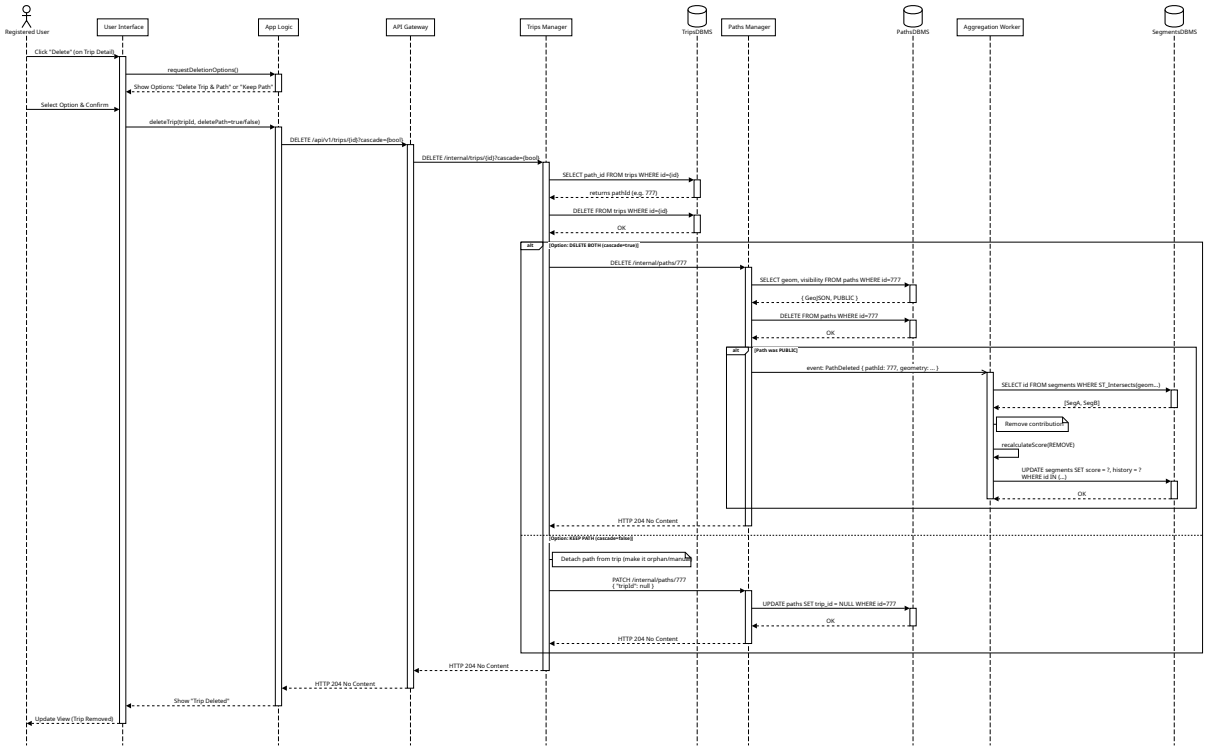


Figure 2.11 – SD9. Delete a Trip

[SD10] Delete a Path

The sequence depicts the control flow for the permanent removal of a User-defined Path. The process is initiated by the User via the *User Interface* and mediated by the *App Logic* and *API Gateway*. Before proceeding with deletion, the *Paths Manager* performs a critical dependency check on its local *Paths DBMS* to ensure referential integrity: it verifies whether the Path is currently linked to an existing Trip (via the `trip_id` foreign key). If dependencies are found, the operation is aborted, and a conflict error is returned to the User.

Conversely, if the Path is an orphan or manually created, the *Paths Manager* proceeds with the deletion. As with the Trip deletion process, if the Path is *Public*, the *Paths Manager* asynchronously emits an event to the *Aggregation Worker*. The worker then recalculates the scores of the affected road segments, removing the User's influence from the aggregated community data.

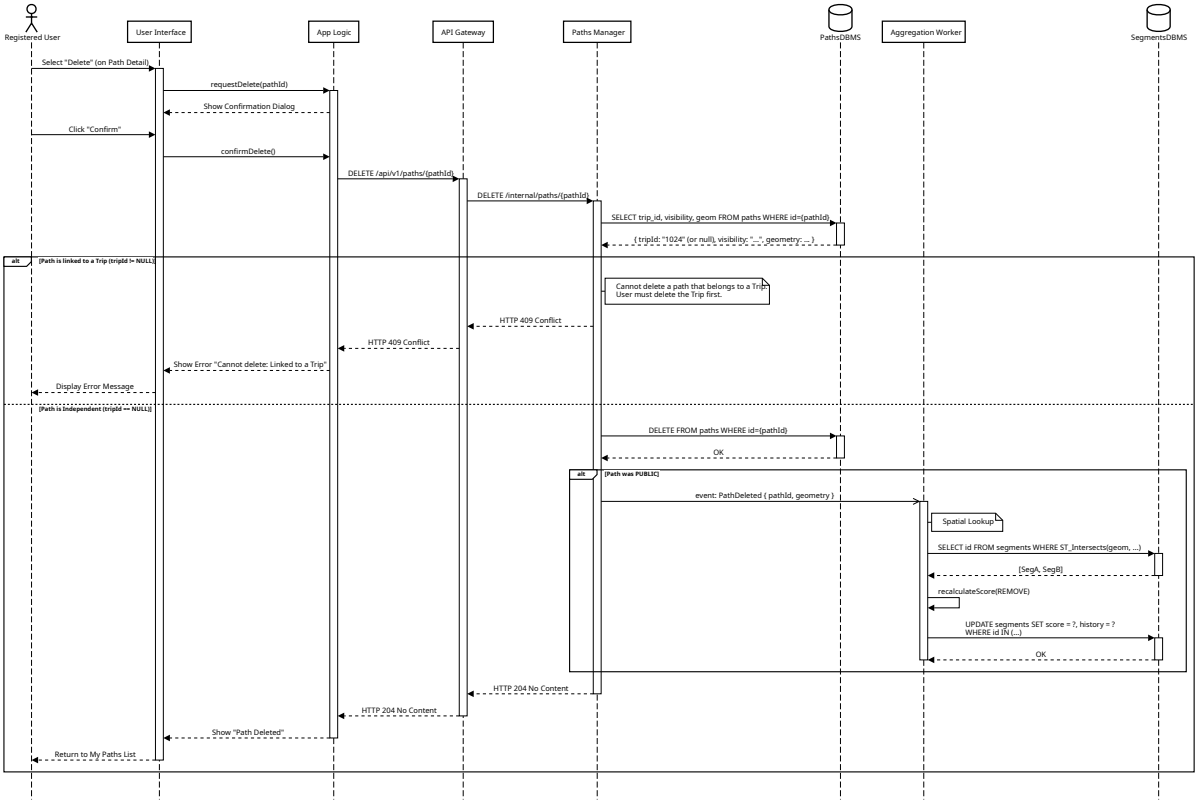


Figure 2.12 – SD10. Delete a Path

2.6 Selected Architectural Styles and Patterns

To address the complexity of the domain and the conflicting requirements of high availability and processing power, the System integrates several well-established architectural patterns.

Microservices and Database-per-Service

The backend adopts a **Microservices Architecture** to ensure loose coupling and independent scalability. Adhering to the *Database-per-Service* pattern, most services (Users, Trips, Auth) maintain exclusive ownership of their private databases. This prevents tight coupling and *integration via database*, allowing each team to evolve their schema independently.

CQRS (Command Query Responsibility Segregation) A strategic deviation from the strict Database-per-Service pattern is applied to the Geospatial Domain to solve the *Read vs. Write* performance tension.

- **Command Side (Write):** The *Aggregation Worker* processes high-volume updates (Write Model) triggered by Users' contributions. It validates and merges data into the *Segments DBMS*.
- **Query Side (Read):** The *Routes Manager* (Read Model) accesses the same dataset but optimized for read-heavy operations (Pathfinding).

This separation allows the System to scale the worker nodes (for heavy processing) independently from the routing nodes (for low-latency queries).

Event-Driven Architecture (Pub/Sub)

To handle long-running tasks without blocking the User Interface, the System implements an asynchronous **Publish/Subscribe** pattern. When a synchronous action completes (e.g., `Path Saved` or `Visibility Changed`), the *Paths Manager* publishes an event to a message broker. The *Aggregation Worker*, acting as a subscriber, picks up the message and performs the recalculation in the background. This ensures the API remains responsive (low latency) while ensuring eventual consistency of the global map.

API Gateway (BFF Pattern)

The **API Gateway** acts as a specialized entry point for the mobile application, implementing the *Backend-for-Frontend (BFF)* pattern. It abstracts the complexity of the internal microservices mesh, providing a single, unified interface to the client. It handles cross-cutting concerns such as:

- **Protocol Translation:** converting external HTTPS requests into internal gRPC calls.
- **Offloading:** handling SSL termination and Authentication verification.
- **Routing:** dispatching requests to the appropriate service instance.

Client-Side Patterns

- **Thick Client:** The mobile application is not a passive view but a computational node. It executes business logic (Motion Analysis) and data validation locally.
- **Offline-First:** The application is designed to function primarily offline during Trip recording. The *Local Persistence Manager* acts as the source of truth for the UI, synchronizing with the server only when network conditions permit (Store-and-Forward).

Resiliency and Structural Patterns

- **Circuit Breaker:** To prevent cascading failures, a Circuit Breaker pattern is implemented in the API Gateway and inter-service calls. If a microservice (e.g., the *Weather Service*) becomes unresponsive or exhibits high latency, the breaker *opens*, immediately returning a fallback response (or cached data) instead of waiting for a timeout.
- **Adapter Pattern:** As described in Section 2.4.3, interaction with external providers (Maps, Weather, Email) is mediated by Adapters. This structural pattern encapsulates the specific API implementation of a vendor (e.g., Google Maps) behind a generic interface (e.g., `IMapService`). This ensures that switching providers does not require refactoring the core domain logic.

2.7 Other Design Decisions

Security and Data Privacy

Given the sensitivity of location data, security is a foundational aspect of the design, addressing data at rest, data in transit, and access control.

Concerning Authentication and Authorization, the System employs **Stateless Authentication** via JSON Web Tokens (JWT):

- **Access Control:** Upon login, the client receives an Access Token containing signed claims (e.g., User ID, Role). This token must be included in the `Authorization` header of every API request. The API Gateway validates the signature at the edge, rejecting unauthorized requests before they reach the microservices.
- **Password Security:** User passwords are never stored in plain text. They are hashed using robust algorithms (such as *Argon2* or *bcrypt*) with per-User salts to prevent rainbow table attacks.

Encryption is also granted:

- **In Transit:** All communication between the Client and the Server, as well as internal cluster traffic, is encrypted using **TLS 1.3** to prevent eavesdropping and Man-in-the-Middle (MitM) attacks.
- **At Rest:** Sensitive data in the database and the backups are encrypted using standard AES-256 encryption.

Persistence Strategy (Polyglot Persistence)

The System rejects the *one-size-fits-all* database approach, opting for a **Polyglot Persistence** strategy to optimize performance based on data characteristics:

- **Spatial Data:** The *Segments DBMS* and *Paths DBMS* use **PostgreSQL with PostGIS extension**. This provides native support for R-Tree indexing and efficient geometric queries, which are critical for routing and map matching.
- **Local Cache:** The mobile client uses **SQLite** for its lightweight footprint and ACID compliance, ensuring reliable offline storage on the device.

Scalability and Orchestration

The backend deployment on Kubernetes allows for fine-grained **Horizontal Scalability**. Since the application logic is stateless (session state is encapsulated in JWTs), the orchestrator can automatically spawn new replicas of specific microservices based on load. For instance, during a high-traffic event, the *Trips Manager* and *Aggregation Worker* pods can be scaled up to handle the influx of data, while the *Users Manager* (used less frequently) remains at baseline capacity.

Chapter 3

User Interface Design

3.1 General Overview

The User Interface of **Best Bike Paths** follows a strict **map-centric approach**, optimized for outdoor usage to ensure high legibility and safe, single-handed interaction while cycling.

The navigation is anchored by a persistent Bottom Navigation Bar, allowing rapid access to core features, while the layout dynamically adapts content based on the User's authentication state. To minimize cognitive load during physical activity, the design adheres to three key principles:

- **Usability and Safety:** High-contrast color schemes and enlarged touch targets are used to facilitate interaction under varying lighting conditions and movement.
- **Data Hierarchy:** The interface clearly separates dynamic performance metrics (Trips) from static geographical data (Paths).
- **Transparency:** Mandatory safety disclaimers are presented upon startup to set clear User expectations.

3.2 User Interface Layouts

3.2.1 Landing Screen

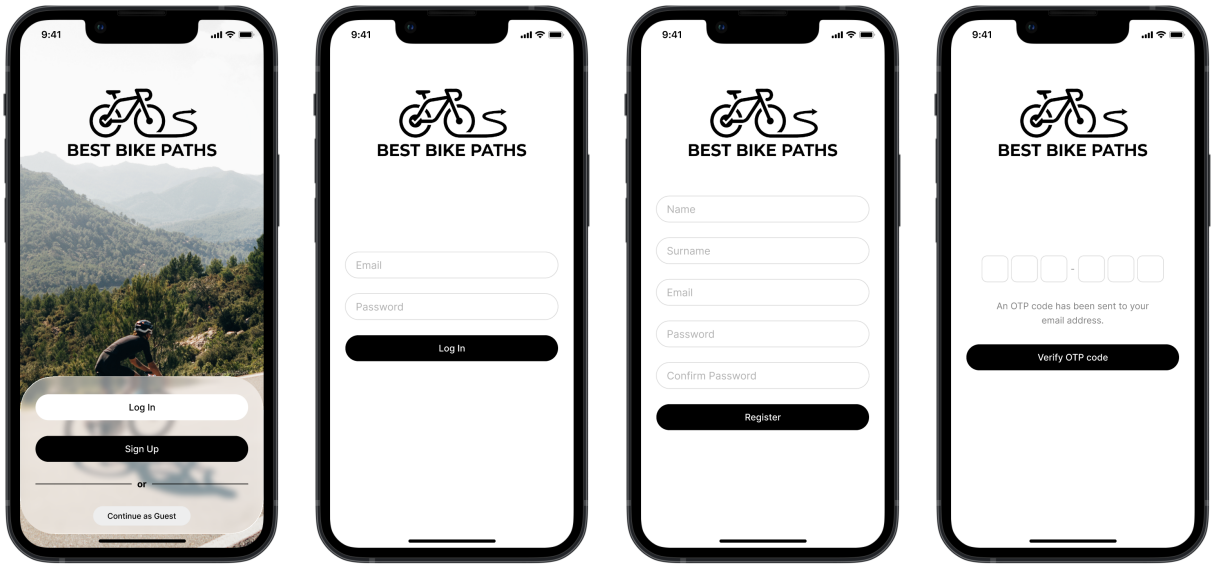


Figure 3.1 – Landing, Sign Up, and Sign In Screens

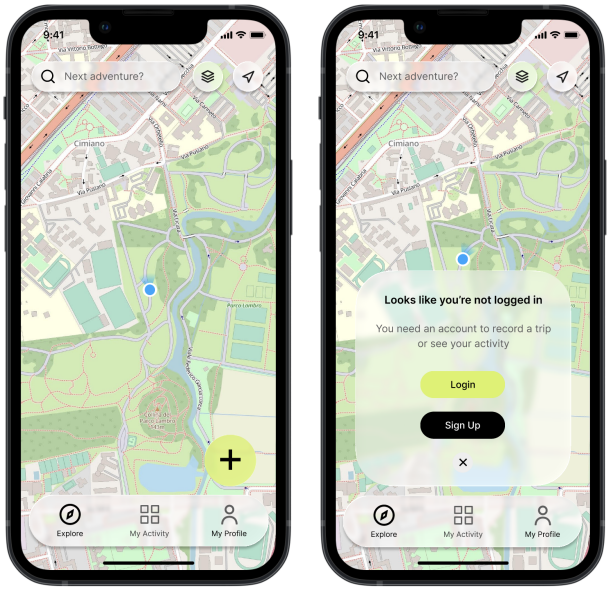
The Landing Screen serves as the application's entry point and primary security gate. Its main purpose is to direct the User toward the appropriate operational mode based on their authentication status, ensuring that access privileges are enforced immediately upon startup.

The interface prioritizes User acquisition by presenting **Sign Up** as the primary call-to-action. This option is designed for new Users who intend to utilize the active features of the application, such as recording Trips, earning statistics, and contributing data to the community. For returning Users who already possess valid credentials, the **Log In** functionality provides access to the authentication process, allowing them to retrieve their personal profile and historical activity data. Additionally, the screen provides a **Guest Access** entry point for unregistered Users, such as tourists or casual explorers.

3.2.2 Explore

The Explore Screen acts as the primary navigational interface for the application. This section is accessible to all User categories, including unauthenticated guests. The interface is dominated by the **Interactive Map**, which shows the User location and their surroundings.

The interface features a prominent **Start Recording** button, typically positioned as a Floating Action Button to ensure easy accessibility. The System implements a role-based behaviour for this control: for **Registered Users**, tapping the button immediately initiates the active recording flow. Conversely, for **Guest Users**, the interaction triggers a modal window informing them that performance tracking is a feature reserved for members, thereby prompting them to sign up to unlock the full potential of the application. This also happens when Guest Users try to navigate to the *My Activity* section.



(a) Explore Section **(b)** Guest User modal window

Figure 3.2 – Explore Section Screens

Search

The search workflow is managed through dedicated interfaces designed to guide the User from route definition to selection.

The **Path Search Input** screen allows Users to define the route's starting point and destination. To streamline the interaction, the interface supports address auto-completion and direct point selection on the map.

Upon submission, the System transitions to the **Search Results** screen, which presents the Pathfinding algorithm's output. The available routes are visualized on the map and listed as summary cards, ranked by their calculated *Score*. Crucially, this view provides more than just distance and duration; each option highlights the aggregated **Path Status** and specific **Warnings** regarding obstacles. This detailed breakdown ensures that Users can evaluate the trade-off between efficiency and safety before starting their journey.



(a) Path Search Input **(b)** Path Search Results

Figure 3.3 – Path Search Screens

3

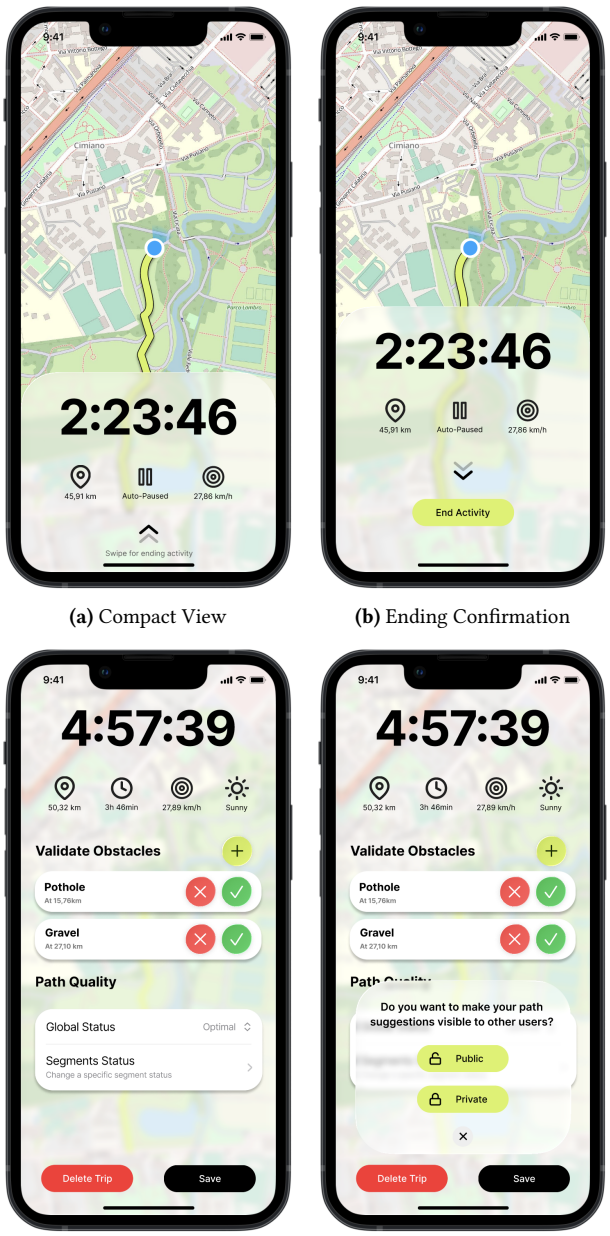
Recording

The tracking functionality is managed through a sequence of interfaces designed to balance ease of use during physical activity with data accuracy during the final review.

The **Active Recording Interface** is optimized for minimal distraction while moving. It features high-contrast typography and large touch targets to display real-time telemetry such as current speed, distance, and duration. A key visual element in this screen is the *Auto-Pause indicator*, which provides immediate feedback when the System detects a stop or non-cycling movement (e.g., walking), ensuring the User is aware of the recording state. To prevent accidental interruptions during the ride, the stop command requires a deliberate interaction (e.g., a *slide-to-confirm* gesture) rather than a simple tap.

Upon concluding the session, the User transitions to the **Post-Ride Validation** screen. This interface is critical for the logic defined in the requirements. Before permanently saving the Trip and the associated Path, the User is presented with a summary of the collected data and is required to perform three validation tasks:

1. **Obstacle Review:** A list of anomalies automatically detected by the sensors (e.g., potholes) is displayed for confirmation or rejection, filtering out false positives. The User can also add undetected obstacles manually.
2. **Path Rating:** A control to assign a qualitative status to the Path and to some specific traversed segments.
3. **Visibility:** Two buttons allowing the User to decide whether to keep the track *Private* or publish it as *Public* to contribute to the global map.



(a) Compact View **(b)** Ending Confirmation

(c) Trip Validation **(d)** Path Visibility

Figure 3.4 – Mockups of Recording Screen

3.2.3 Profile

The Profile Section serves as the central hub for managing the User's identity and application preferences. For a **Registered User**, this interface acts as a personal dashboard displaying the account details defined in the domain model, such as name and surname.

Beyond simple data visualization, the screen allows access to the **App Settings** submenu. Here, Users can configure technical parameters and manage privacy controls, including permissions for background sensor usage. In strict compliance with GDPR requirements, this section houses critical data management features, granting Users the ability to export their personal data or request the permanent deletion of their account and associated logs.

The interface dynamically adapts its content based on the current session state. While registered Users are presented with a **Logout** option to securely terminate their session, **Guest Users** accessing this section view a restricted interface. Instead of personal details, they are presented with prompts to **Log In** or **Sign Up**, encouraging the transition from a transient anonymous session to a persistent account to unlock full platform capabilities.

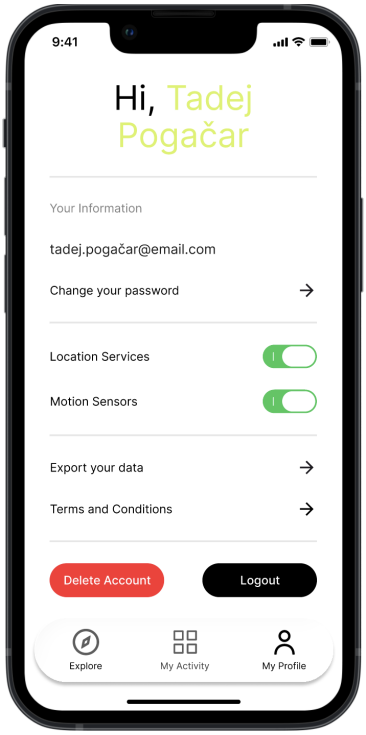


Figure 3.5 – Profile Screen

3.2.4 My Activity

The **My Activity** section acts as the comprehensive logbook for the Registered User, consolidating all historical data generated within the platform. Accessible directly from the main bottom navigation bar, this screen serves as the entry point for managing personal content.

The interface is designed around a clear separation of concerns, subdividing the screen into two sections to allow Users to have a glance at their activity:

- **Trips:** For accessing the chronological history of recorded performances.
- **Paths:** For managing the library of saved geographical Paths.

This layout ensures that Users can rapidly switch between reviewing their athletic performance and managing their map contributions without losing navigation context. By default, the screen opens on the most recent activity list, providing immediate access to the latest data.

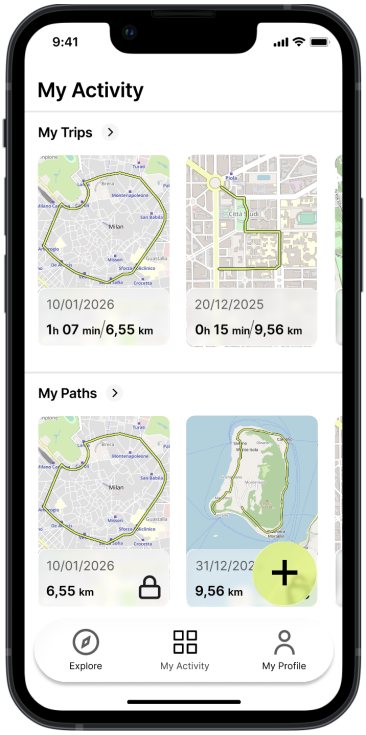


Figure 3.6 – My Activity Screen

3

Trip Details

Upon selecting a specific entry from the history list, the User is presented with the **Trip Detail** view. This interface offers a comprehensive post-ride analysis, visualizing the recorded track overlaid on the interactive map to allow Users to review the exact route taken.

Below the geospatial representation, a dedicated dashboard displays the aggregated performance metrics captured during the session, including total distance, duration, average speed, and elevation gain. To provide a complete context for the athletic activity, the screen also presents the meteorological data (e.g., temperature and weather condition) retrieved from external services for that specific time and location. From this interface, the User retains full control over their data, with the specific option to permanently delete the recording from the System if deemed unnecessary.



(a) Compact View **(b)** Expanded View

Figure 3.7 – Trip Details Screens

Path Details

Accessing an item from the Path Library opens the **Path Details** view, which focuses on the static geographical definition of the route. Unlike the Trip view, this interface highlights the qualitative attributes of the road network. The Path is rendered on the map, colour-coded according to its **Aggregated Status**, and accompanied by its calculated *Reliability Score*.

A central control in this interface is the **Visibility Button**. This feature empowers the User to manage the lifecycle of the data: they can choose to keep the Path *Private* for personal use or switch it to *Public*. Publishing the Path triggers the System's data aggregation algorithms, merging the User's local ratings with the global map to assist the entire community. Finally, the interface provides a specific option to delete the Path entity, ensuring the User retains full ownership of their content.

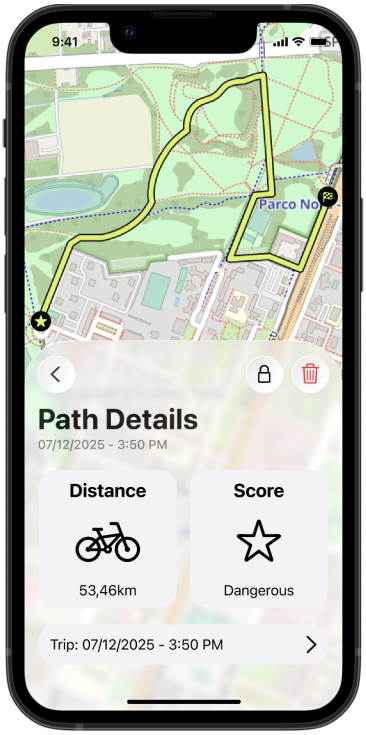


Figure 3.8 – Path Details Screen

Manual Path Creation

The **Manual Path Creation** interface provides a dedicated toolset for Registered Users to contribute to the map inventory without the need for real-time tracking. This feature is particularly designed for expert Users who wish to map known trails or plan future routes from the comfort of their home.

The workflow consists of two main stages. First, the User interacts with the **Track Editor**, selecting a sequence of waypoints on the map. The System automatically snaps these points to the underlying road network to reconstruct a valid, continuous geometry.

Once the route is defined, the interface transitions to the **Details Configuration** form. Here, the User assigns a global status to the Path to warn future cyclists. Advanced controls allow for granular overrides, enabling the User to mark specific segments with a different status if conditions vary along the route. Finally, the User sets the initial visibility permissions before saving the new entry to the database.



(a) Create a Path **(b)** Confirm a Path

Figure 3.9 – Path Creation Screens

Chapter 4

Requirements Traceability

4.1 Functional Requirements Traceability

The following table maps the Functional Requirements (defined in the RASD) to the Design Components (defined in the DD) that are responsible, fully or partially, for their satisfaction.

Component	Satisfied Requirements
Sensor Controller	**[R6]** Whenever a Trip is being recorded, the System shall keep track of the Path followed by the User acquiring GPS coordinates. **[R7]** Whenever a Trip is being recorded, the System shall analyse data from the device's accelerometer and gyroscope to automatically detect potential obstacles (e.g., potholes).
Motion analyser	**[R7]** Whenever a Trip is being recorded, the System shall analyse data from the device's accelerometer and gyroscope to automatically detect potential obstacles (e.g., potholes). **[R9]** Whenever a Trip is being recorded, the System shall automatically pause the data acquisition if the detected speed or movement pattern suggests the User is not cycling (e.g., walking or stationary).
Trip Recording Manager	**[R4]** Upon request, the System shall allow a registered User to start a training session. **[R5]** Upon request, the System shall allow a registered User to end a training session, as long as they have already started it. **[R6]** Whenever a Trip is being recorded, the System shall keep track of the Path followed by the User acquiring GPS coordinates. **[R8]** Whenever the System detects a potential obstacle during a recording, it shall temporarily store its location and type for validation at the end of the session. **[R9]** Whenever a Trip is being recorded, the System shall automatically pause the data acquisition if the detected speed or movement pattern suggests the User is not cycling (e.g., walking or stationary). **[R23]** Whenever a Trip is recorded, the System shall elaborate statistics on performance, such as estimated speed, total length, and time. **[R24]** Whenever a Trip is concluded and external weather services are available, the System shall retrieve and store the weather conditions associated with the Trip's location and time.
Aggregation Worker	**[R19]** Whenever a Path is made public, the System shall recalculate the status of the involved street Segments by aggregating the User's data with existing community data. **[R20]** When aggregating conflicting data about a Path Segment, the System shall prioritize information based on its freshness (recency of submission). **[R21]** When aggregating conflicting data about a Path Segment, the System shall resolve inconsistencies by considering the number of confirming reports from different Users. **[R22]** Whenever a public Path is made private or deleted, the System shall remove the specific User's contribution from the aggregated community data, triggering a recalculation of the Segment's status.
Trips Manager	**[R14]** Upon request, the System shall allow a registered User to delete their recorded Trips. **[R18]** Upon request, the System shall allow a registered User to view all their Trips and Paths, whether they are private or public.

Component	Satisfied Requirements
Paths Manager	**[R12]** The System shall prevent any modification to the Path's structural data (GPS track) and descriptive details (status, obstacles) once the Path has been saved to the System. **[R13]** Upon request, the System shall allow a registered User to insert a Path manually, as long as a list of streets and statuses is provided. **[R15]** Upon request, the System shall allow a registered User to delete their Paths, whether inserted manually or automatically. **[R16]** Upon request, the System shall allow a registered User to make their Path available to the community (public), as long as said Path is currently private. **[R17]** Upon request, the System shall allow a registered User to make their Path private, as long as said Path has already been shared with the community. **[R18]** Upon request, the System shall allow a registered User to view all their Trips and Paths, whether they are private or public.
App Logic	**[R1]** Upon request, the System shall allow a User to sign up to the Platform, as long as they provide all the required information and a profile is not already associated with it. **[R2]** Upon request, the System shall allow a User to log in to the Platform, as long as the correct credentials are provided. **[R3]** Upon request, the System shall allow a User to access the Platform as a guest. **[R10]** Upon concluding a recorded Trip (and before final storage), the System shall require the User to validate (confirm or delete) the automatically detected obstacles and assign a general status to the traversed Segments. **[R11]** Upon concluding a recorded Trip (and before final storage), the System shall allow the User to manually insert and classify obstacles that were not automatically detected by the sensors. **[R13]** Upon request, the System shall allow a registered User to insert a Path manually, as long as a list of streets and statuses is provided. **[R14]** Upon request, the System shall allow a registered User to delete their recorded Trips. **[R15]** Upon request, the System shall allow a registered User to delete their Paths, whether inserted manually or automatically. **[R16]** Upon request, the System shall allow a registered User to make their Path available to the community (public), as long as said Path is currently private. **[R17]** Upon request, the System shall allow a registered User to make their Path private, as long as said Path has already been shared with the community. **[R18]** Upon request, the System shall allow a registered User to view all their Trips and Paths, whether they are private or public. **[R25]** Upon request, the System shall provide a list of Paths connecting two locations to any User (registered or guest), visualizing them on a map ordered by their calculated score.
Authentication Manager	**[R1]** Upon request, the System shall allow a User to sign up to the Platform, as long as they provide all the required information and a profile is not already associated with it. **[R2]** Upon request, the System shall allow a User to log in to the Platform, as long as the correct credentials are provided. **[R3]** Upon request, the System shall allow a User to access the Platform as a guest.
Routes Manager	**[R25]** Upon request, the System shall provide a list of Paths connecting two locations to any User (registered or guest), visualizing them on a map ordered by their calculated score. **[R26]** Whenever providing search results, the System shall calculate a Path's score based on its aggregated status (e.g., optimal vs. bad) and its effectiveness in reaching the destination.

Component	Satisfied Requirements
Users Manager	**[R1]** Upon request, the System shall allow a User to sign up to the Platform, as long as they provide all the required information and a profile is not already associated with it. **[R18]** Upon request, the System shall allow a registered User to view all their Trips and Paths, whether they are private or public.

Table 4.1 – Functional Requirements Traceability

4

4.2 Non-Functional Requirements Traceability

The following tables map the Non-Functional Requirements (Performance and System Attributes) defined in the RASD to the architectural decisions and components detailed in Chapter 2 of this document.

4.2.1 Performance Requirements

The performance requirements outlined in Section 3.3 of the RASD are addressed through some design decisions (already discussed in Section 2.6). Below a table of the main requirements and how they are achieved.

RASD Requirement	Design Justification
Response Time for Interactive Tasks (Standard interactions $<$ 1 sec)	The architecture minimizes latency for standard operations through the **Database-per-Service** pattern. By strictly isolating the User and Authentication databases from the heavy geospatial datasets, the System ensures that simple queries (like login or profile retrieval) never contend for resources with complex analytical tasks. Additionally, the API Gateway provides a unified and optimized entry point.
Path Search Latency (Routing suggestions $<$ 5 sec for 95% of requests)	To handle the computational complexity of routing, the System implements the **CQRS Pattern**. The *Routes Manager* accesses a dedicated Read Model hosted on a Spatial DBMS optimized with R-Tree indices. This decoupling ensures that heavy write operations from incoming Trips do not lock the database tables needed for pathfinding, allowing for sub-second geometric queries.
Sensor Processing Latency (Client-side processing $<$ 200 ms)	The **Thick Client** architecture moves the processing logic to the *Edge*. The *Motion Analyser* component runs directly on the User's device, processing accelerometer data in real-time without any network overhead. This ensures that obstacle detection happens instantaneously, independent of server load or connectivity status.
Data Aggregation Freshness (Global update $<$ 10 min)	The System adopts an **Event-Driven** approach for data consistency. The *Aggregation Worker* processes updates asynchronously via a message queue. Combined with **Horizontal Auto-scaling**, the infrastructure can automatically spawn additional worker instances to process bursts of reports in parallel, ensuring the global map is updated well within the 10-minute constraint.
System Capacity (Support 10,000 concurrent Users)	Scalability is achieved through the use of **Stateless Microservices**. Since User sessions are encapsulated in self-contained JWTs rather than stored in server memory, the Kubernetes orchestrator can distribute traffic across any number of replicas. The API Gateway acts as a Load Balancer to ensure no single instance becomes a bottleneck.
Synchronization Latency (1-hour Trip upload $<$ 30 sec)	The **Local Persistence Manager** pre-aggregates and structures the data on the device before transmission, reducing the granularity of the payload. Furthermore, the use of efficient serialization formats (JSON over gRPC internally) ensures that the bulk upload of the Trip log is optimized for throughput.

Table 4.3 – Performance Requirements

4.2.2 Software System Attributes

This table maps the quality attributes, described in Section 3.5 of the RASD, to the structural patterns and security measures adopted.

RASD Attribute	Design Justification
Reliability	Reliability is addressed with a dual strategy. For the critical phase of **Trip Recording**, data integrity is guaranteed by local buffering (*Local Persistence Manager*), preventing data loss even if the network fails during the activity. For all other interactive features (which assume connectivity), reliability is ensured by the **Redundancy** of backend services (Kubernetes replicas) and the use of **Circuit Breakers** to handle transient failures of external providers gracefully.
Availability	High availability is achieved through the **Redundancy** provided by the Kubernetes orchestration layer. Critical microservices (Authentication, Trips, Users) are deployed with multiple replicas behind the API Gateway's Load Balancer. If a specific instance fails, traffic is seamlessly rerouted to a healthy one. Furthermore, **Horizontal Auto-scaling** ensures the System remains responsive even during unexpected traffic spikes.
Security	Security is enforced at multiple layers. The **API Gateway** acts as a centralized enforcement point for SSL/TLS termination, protecting data in transit. Access control relies on **Stateless Authentication** via signed JWTs, ensuring that no User credentials are exchanged after the initial login. Additionally, sensitive data stored in the databases is protected using industry-standard AES-256 encryption.
Maintainability	The adoption of a **Microservices Architecture** combined with the **Database-per-Service** pattern ensures high cohesion and loose coupling. This allows development teams to update, refactor, or scale individual functional domains (e.g., changing the Routing algorithm) without impacting other parts of the System. The use of the **Adapter Pattern** for external services further simplifies maintenance by isolating vendor-specific logic.
Portability	The System maximizes portability on both tiers. On the Client Side, the application is developed using a **Cross-Platform Framework** (e.g., Flutter). This allows a single codebase to be compiled into native artifacts for both iOS and Android, ensuring consistent behaviour across different ecosystems. On the Server Side, **Containerization (Docker)** ensures the backend logic runs identically on any infrastructure (Cloud or On-Premise) supporting a container runtime.

Table 4.5 – Software System Attributes

Chapter 5

Implementation, Integration and Test Plan

5.1 Overview

The purpose of this section is to define the systematic strategy for the implementation, integration, and validation. Transitioning from the architectural design to the final software release requires a structured roadmap to effectively manage the complexity of the distributed environment, specifically the interaction between the *Thick Client* mobile application and the backend *Microservices cluster*.

The implementation process is organized into logical phases, or *Waves*, and follows a **bottom-up approach**. This strategy prioritizes the establishment of the data persistence layer and critical infrastructural services, such as the *API Gateway* and *Authentication Service*. Subsequently, the development focuses on the core business logic, before finalizing with the implementation of the Mobile User Interface and the integration of external services. This dependency-driven roadmap ensures that each phase produces a testable and consistent increment of the system.

Parallel to the implementation, the integration strategy is designed to verify the correct interplay between components. Given the decoupled nature of the architecture, integration activities focus on two critical boundaries: the synchronous RESTful communication between the Mobile App and the API Gateway, and the asynchronous event-driven interactions between internal microservices.

Lastly, the Test Plan adopts a multi-level approach, ranging from unit testing of individual modules to end-to-end system testing, verifying both the functional correctness and the adherence to non-functional constraints such as performance and reliability.

5.2 Implementation Plan

Wave 1: Core Infrastructure and Identity

The first phase focuses on establishing the communication backbone and the User management capabilities.

- **Backend Infrastructure:** Setup of the Kubernetes Cluster, configuration of the API Gateway (Ingress Controller, SSL termination), and the Message Broker (e.g., RabbitMQ/Kafka) for asynchronous events.
- **Databases:** Provisioning of the persistent storage, specifically the PostgreSQL instances for Users and Security data.
- **Authentication & Users Services:** Implementation of the `Authentication Service` (JWT generation, OTP logic via Email Adapter) and `Users Service` (Profile management).
- **Client Skeleton:** Setup of the Mobile App project (Flutter), implementation of the UI navigation shell, and integration of the Local SQLite database structure.

Wave 2: Tracking and Data Collection

This phase addresses the core value proposition: recording Trips and saving them.

- **Mobile Sensor Logic:** Implementation of the *Sensor Controller* and *Motion Analyser* modules within the mobile app to handle raw GPS and accelerometer streams.
- **Local Persistence:** Implementation of the logic to store Trips offline in the local SQLite DB.

- **Trips & Paths Services:** Development of the backend microservices responsible for data ingestion. This includes defining the internal gRPC contracts for the communication between *Trips Manager* and *Paths Manager*.
- **Spatial Database:** Setup of the PostGIS extension to handle geometry storage.

Wave 3: Analytics and Routing

The final phase implements the complex algorithms that leverage the data collected in Wave 2.

- **Aggregation Logic:** Development of the *Aggregation Worker*. This component is implemented to listen to *Path Saved* events and update the global *Reliability Scores*.
- **Routing Engine:** Implementation of the *Routes Manager*, optimizing the pathfinding algorithm on the Read-Model (Segments DBMS).
- **External Integration:** Finalizing the adapters for third-party services (Weather and Maps) to enrich the User experience.

5.3 Integration Strategy

Instead of a generic layer-based integration, a **Feature-Oriented Integration Strategy** is adopted. Since the System is composed of autonomous microservices and a heavy Client, integration testing is performed by assembling the specific components required to satisfy a complete functional scenario.

Below, the integration flow is detailed for each primary feature of the system.

Feature A: User Onboarding (Sign Up & Login)

This integration step verifies the security chain and the correct propagation of identity.

- **Components Involved:** Mobile App, API Gateway, Authentication Service, Users Service, Email Provider.
- **Integration Logic:** The Mobile App is integrated with the API Gateway to ensure REST requests are correctly routed. The critical internal integration point is between the *Auth Service* and the *Users Service*: when a User validates the OTP, the Auth Service must successfully invoke the internal gRPC endpoint of the Users Service to create the persistent profile. Finally, the ability to send emails is verified by integrating the SMTP Adapter.

Feature B: Offline Trip Recording

This step validates the *Thick Client* architecture, focusing purely on local component integration without backend dependency.

- **Components Involved:** Sensor Controller, Motion Analyser, Local Persistence Manager (SQLite).
- **Integration Logic:** The integration focuses on the data pipeline within the device. The *Sensor Controller* is analysed to see if it correctly streams hardware data to the *Motion Analyser*, and the Analyser's output (anomalies/coordinates) is checked to see if it is correctly serialized and written to the *Local Persistence Manager*. This ensures the *Offline-First* requirement is met.

Feature C: Trip Synchronization (Save)

This is the most complex integration scenario, bridging the Client and multiple Backend services.

- **Components Involved:** Mobile App, Trips Manager, Paths Manager, Trips DBMS, Paths DBMS.
- **Integration Logic:** The Mobile App integrates with the backend via the `POST /save` endpoint. On the server side, the transactional integrity is verified: the *Trips Manager* must save the metadata and synchronously call the *Paths Manager* (via gRPC) to save the geometry. The integration is considered successful only if both databases (Trips DBMS and Paths DBMS) end up in a consistent state.

Feature D: Global Map Consistency (Aggregation)

This step integrates the asynchronous Event-Driven architecture.

- **Components Involved:** Paths Manager, Message Broker, Aggregation Worker, Segments DBMS.
- **Integration Logic:** The integration point is the Message Broker (Pub/Sub). The purpose is to verify that when the *Paths Manager* commits a new public Path, an event is correctly published. The *Aggregation Worker* must then consume this message, retrieve the geometry, and execute the merging algorithm on the *Segments DBMS*. This validates the eventual consistency of the system.

Feature E: Routing and Exploration

This final step integrates the Read-Model components.

- **Components Involved:** Mobile App, Routes Manager, Segments DBMS (Read Access).
- **Integration Logic:** The Mobile App requests a route calculation. The integration verifies that the *Routes Manager* can efficiently query the *Segments DBMS* (populated by Feature D) using spatial indices and return a valid GeoJSON response to the client within the defined latency constraints.

5.4 Test Plan

To ensure the reliability, performance, and usability of the *Best Bike Paths* System, testing is performed at multiple levels of granularity. The strategy follows the *Test Pyramid* principle, starting from isolated components and moving up to complete end-to-end scenarios and user acceptance.

Unit Testing

The first line of defence focuses on verifying the internal logic of individual components in isolation.

- **Backend Microservices:** Each service (Authentication, Trips, Routes) is tested using frameworks like *JUnit* or *PyTest*.
- **Mobile App Logic:** The Client-side logic is tested using *Flutter Test*. Critical tests include:
  - **Motion Analyser:** Feeding pre-recorded raw sensor datasets (accelerometer/GPS) to the analyser to verify it correctly detects potholes and stops recording when the User is stationary.
  - **Local Persistence:** Verifying that Trips are correctly saved to SQLite and that the sync queue is managed properly.

Integration Testing

This phase verifies that different components interact correctly. Given the Microservices architecture, this is the most critical technical testing phase.

- **Inter-Service Communication:** The purpose is to verify that internal gRPC calls handle payloads and errors correctly (including timeouts and circuit breaker activation).
- **Database Integration:** The purpose is to verify that the *Spatial DBMS* (PostGIS) correctly handles geometric queries and that transactions are atomic.

System Testing (End-to-End)

These tests validate the complete flow of Use Cases, treating the system as a black box. Automated scripts (using tools like *Selenium* or *Appium*) simulate user behaviour.

Performance and Stress Testing

These tests are aimed at verifying that the Non-Functional Requirements are satisfied.

- **Load Testing:** 10,000 concurrent virtual Users hitting the API Gateway are simulated to verify that the Kubernetes *Horizontal Pod Autoscaler* triggers correctly, spawning new replicas of the target service.
- **Chaos Engineering:** Intentionally kill random pods in the staging environment to ensure the *Circuit Breaker* opens and the mobile app degrades gracefully instead of crashing.

User Acceptance Testing (UAT)

The final phase involves real human interaction to validate usability and real-world performance.

- **Alpha Testing (Internal):** The development team and internal stakeholders install the app on real devices to catch obvious bugs and UI glitches.
- **Beta Testing (Closed Public):** A limited group of external users (e.g., 50 cyclists) is invited via *TestFlight* (iOS) or *Google Play Console* (Android). This phase is crucial to:
  - Collect real sensor data from diverse hardware to tune the *Motion Analyzer* sensitivity.
  - Identify edge cases in GPS accuracy (e.g., tunnels, high-rise urban canyons).

Chapter 6

Effort Spent

Author	Section 1	Section 2	Section 3	Section 4	Section 5
Daniele Toniolo	0.5	20	15	4	0.5
Stefano Ungaro	1	20	8	2	4
Matteo Zappa	0.5	9	22	0	1

Table 6.1 – Effort spent by each team member (unit is hours)

Chapter 7

References

7.1 Used Tools

- **Overleaf**[1]: LaTeX text editor for collaborative writing.
- **draw.io**[2]: creation of UML diagrams.
- **SequenceDiagram.org**[3]: creation UML Sequence Diagrams.
- **Figma**[4]: UI creation.
- **Google Gemini 3 Pro**[5]: used for grammar, style, and coherence checks. Prompts included specific sections of this document with instructions to analyse and refine the text based on academic standards and logical consistency.

[1] https://www.overleaf.com
[2] https://www.draw.io
[3] https://www.sequencediagram.org/
[4] https://www.figma.com
[5] https://gemini.google.com

Appendix A

Detailed Diagrams

This appendix presents the high-resolution, full-page versions of the diagrams discussed in previous chapters. These enlarged figures are provided to ensure optimal readability and to allow for a detailed inspection of the System's components, deployment nodes, and sequence logic. Here is a summary of the diagrams included in this appendix.

Image	Description	Original Figure
Figure A.1	UML component diagram	Figure 2.1
Figure A.2	Deployment view	Figure 2.2
Figures A.3 to A.4	SD1 sequence: Sign Up by a User	Figure 2.3
Figure A.5	SD2 sequence: Log In by a User	Figure 2.4
Figure A.6	SD3 sequence: Access as Guest	Figure 2.5
Figures A.7 to A.8	SD4 sequence: Record a Trip	Figure 2.6
Figure A.9	SD5 sequence: Validate a Trip and a Path	Figure 2.7
Figure A.10	SD6 sequence: Change Path's Visibility	Figure 2.8
Figures A.11 to A.12	SD7 sequence: Create a Path Manually	Figure 2.9
Figure A.13	SD8 sequence: Search for a Route	Figure 2.10
Figure A.14	SD9 sequence: Delete a Trip	Figure 2.11
Figure A.15	SD10 sequence: Delete a Path	Figure 2.12

Table A.1 – Reference table for detailed diagrams in the appendix

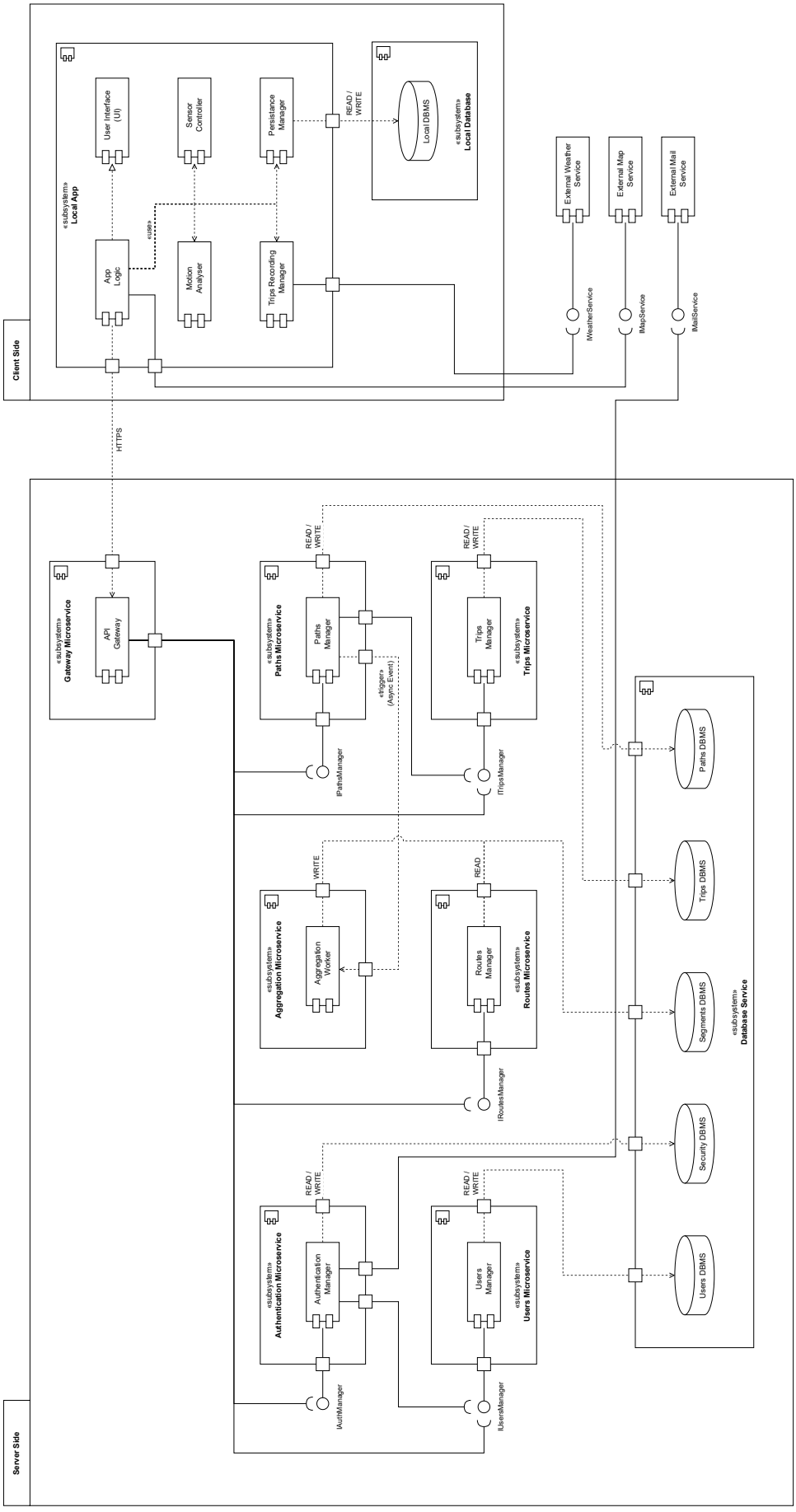


Figure A.1 – Full page UML Components Diagram.

A

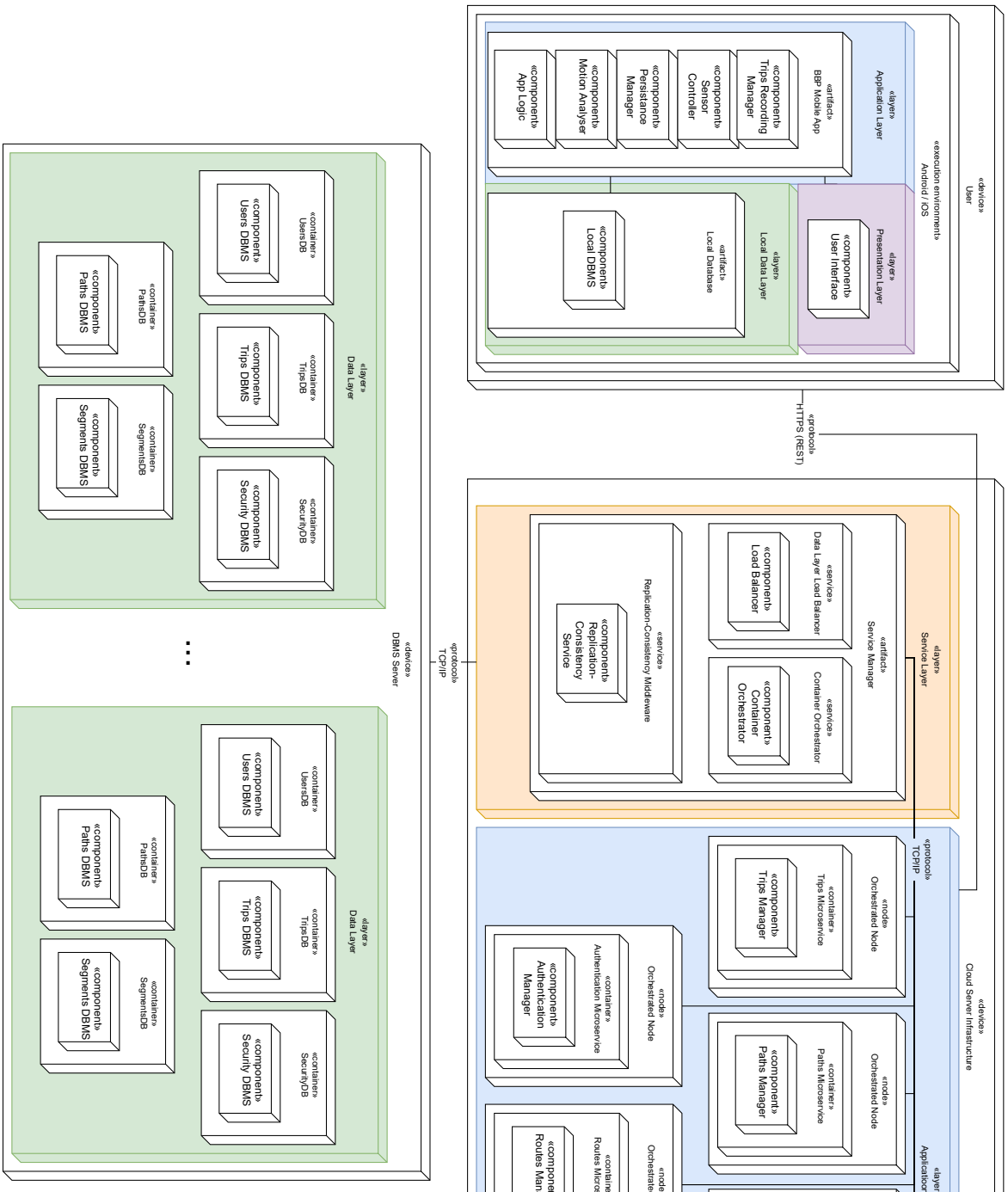


Figure A.2 – Full page Deployment View of Best Bike Paths System

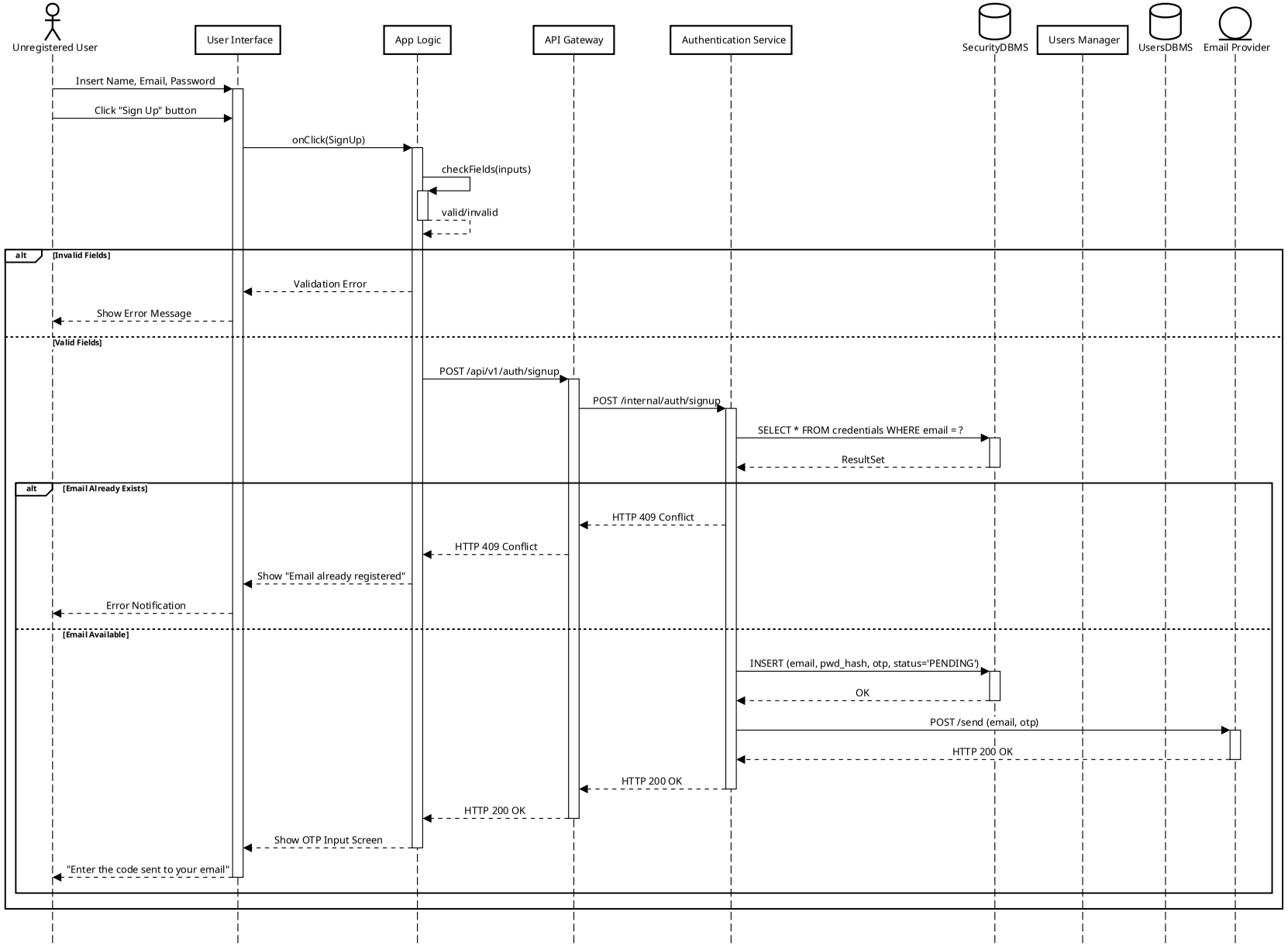


Figure A.3 – SD1. Full page diagram (part 1)

A

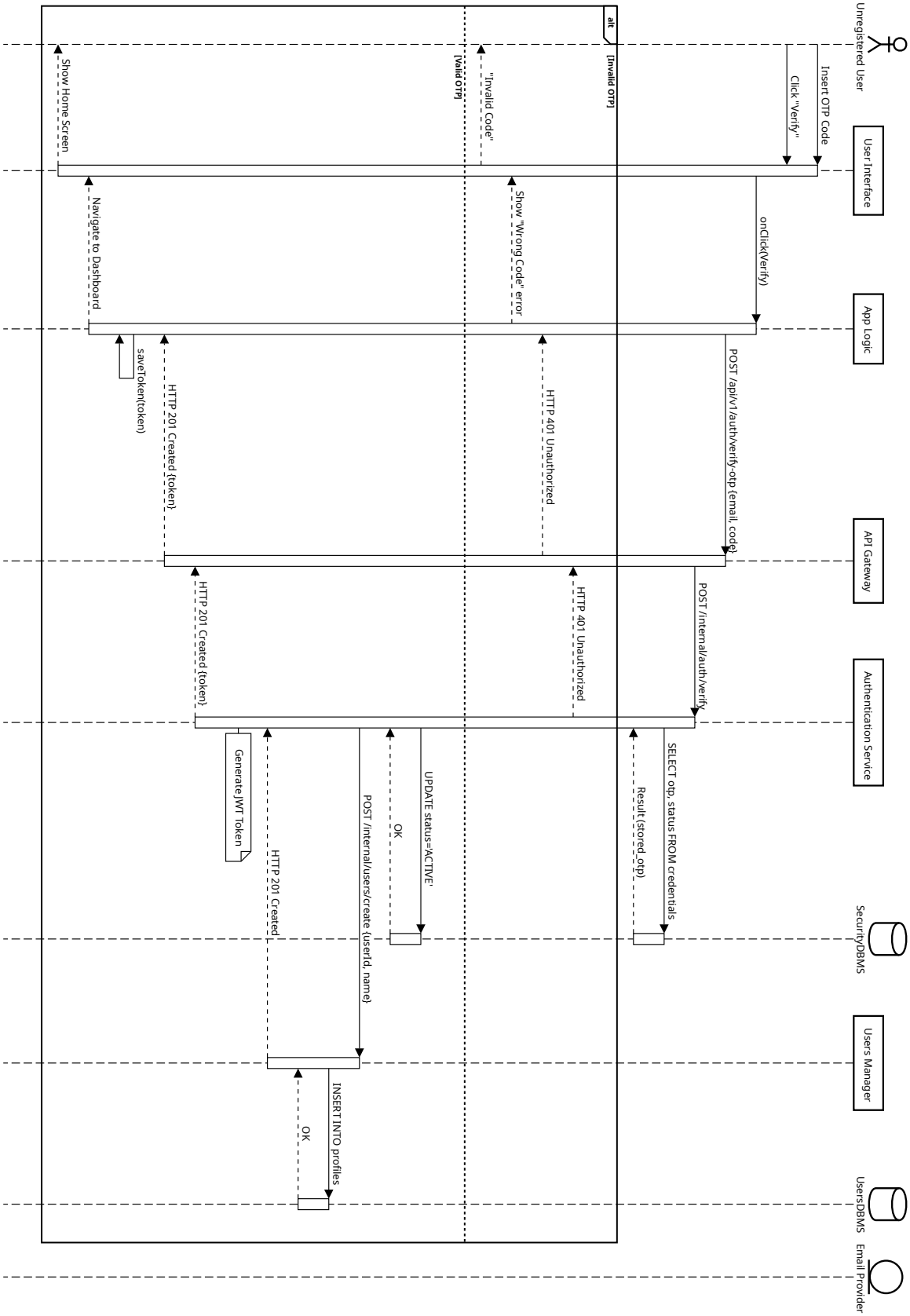


Figure A.4 – SD1. Full page diagram (part 2)

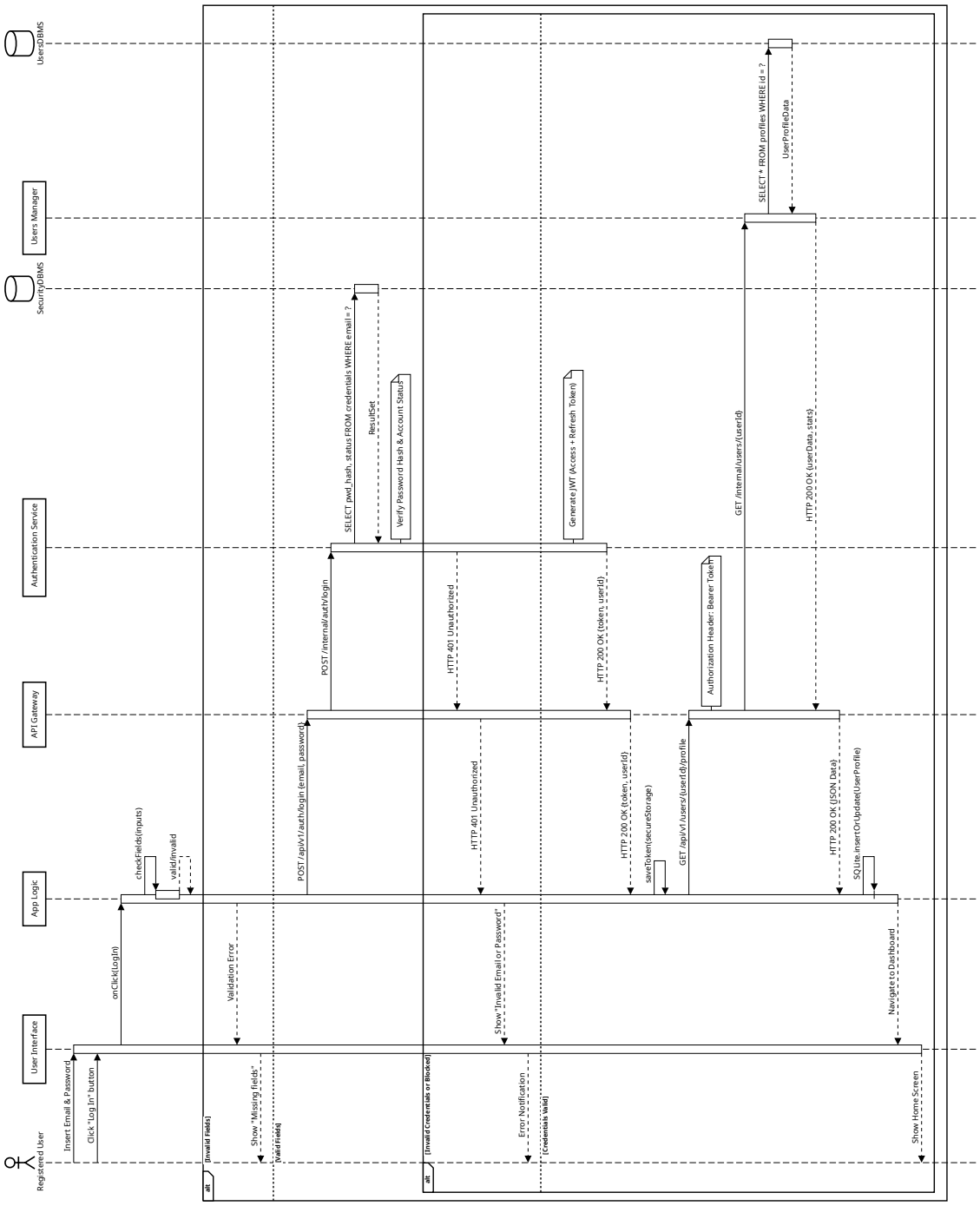


Figure A.5 – SD2. Full page diagram

A

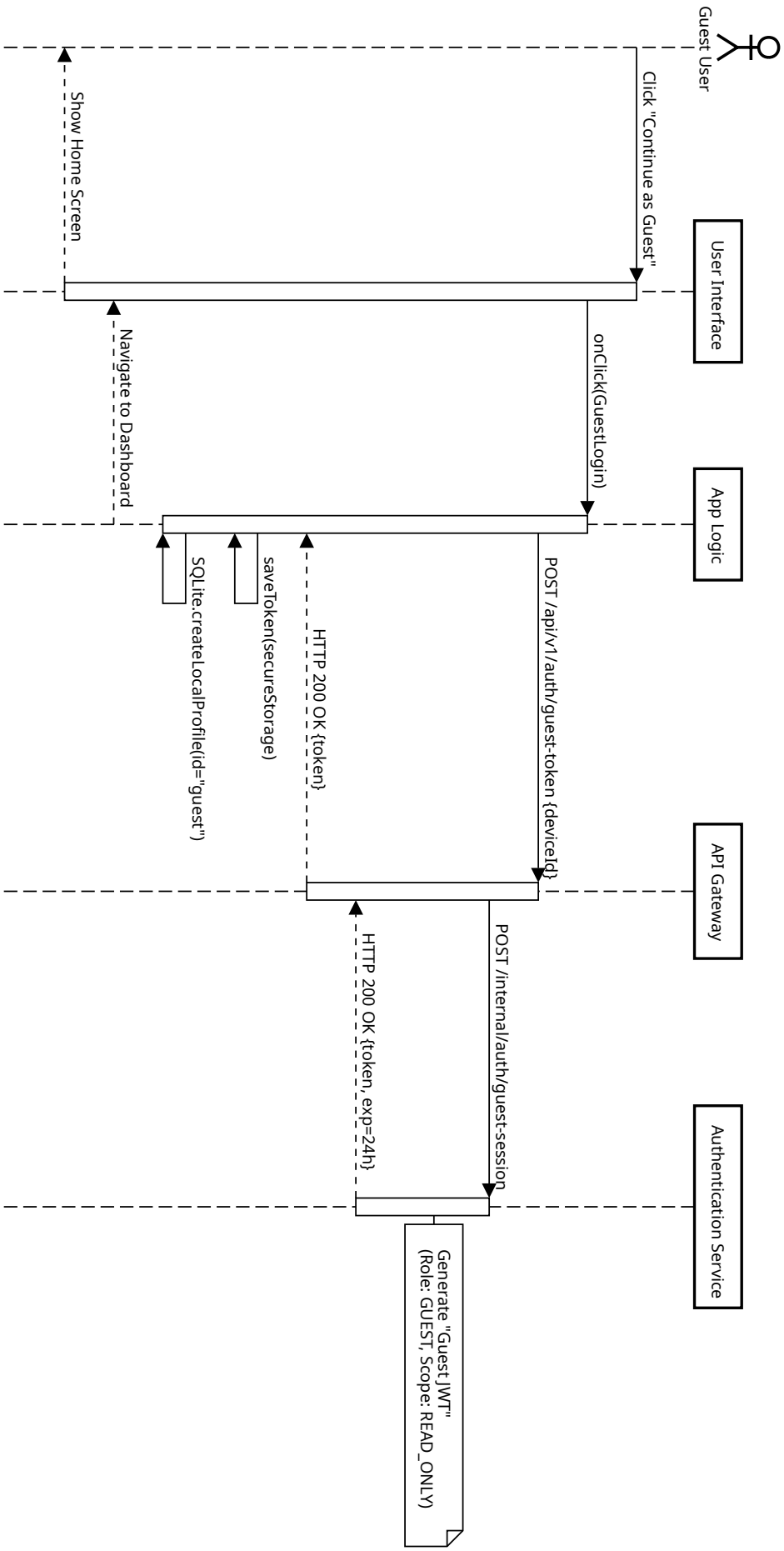


Figure A.6 – SD3. Full page diagram

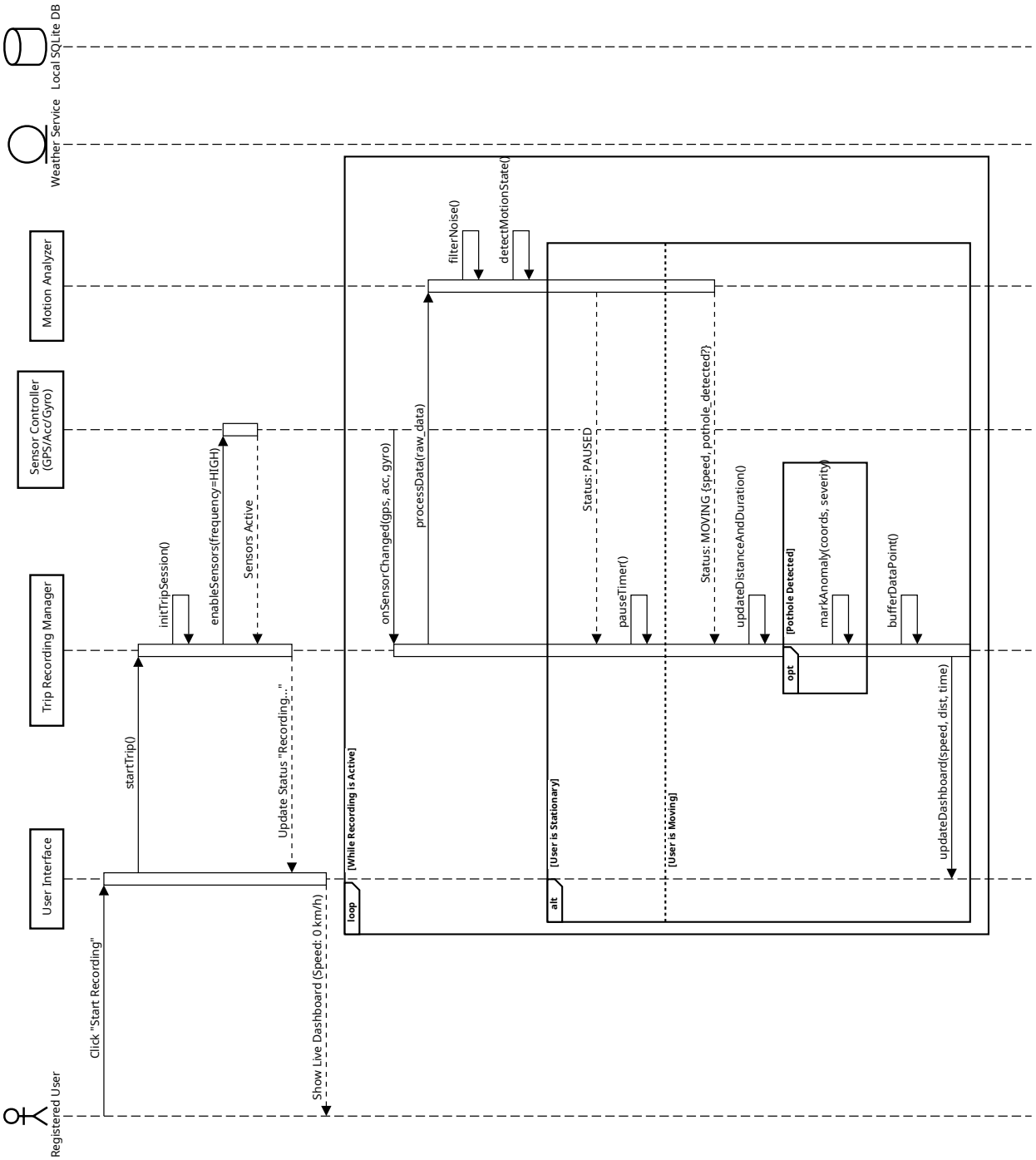


Figure A.7 – SD4. Full page diagram (part 1)

A

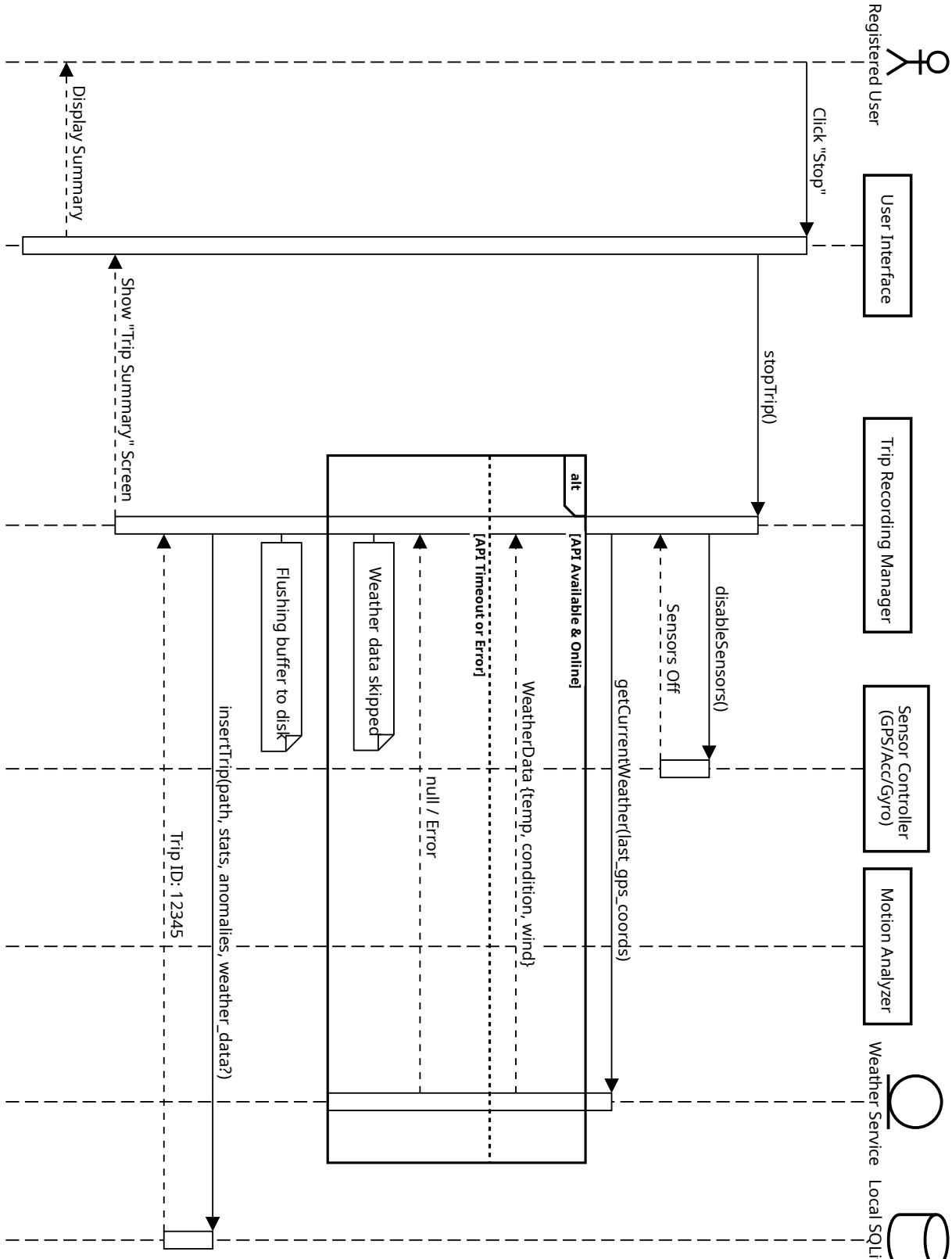


Figure A.8 – SD4. Full page diagram (part 2)

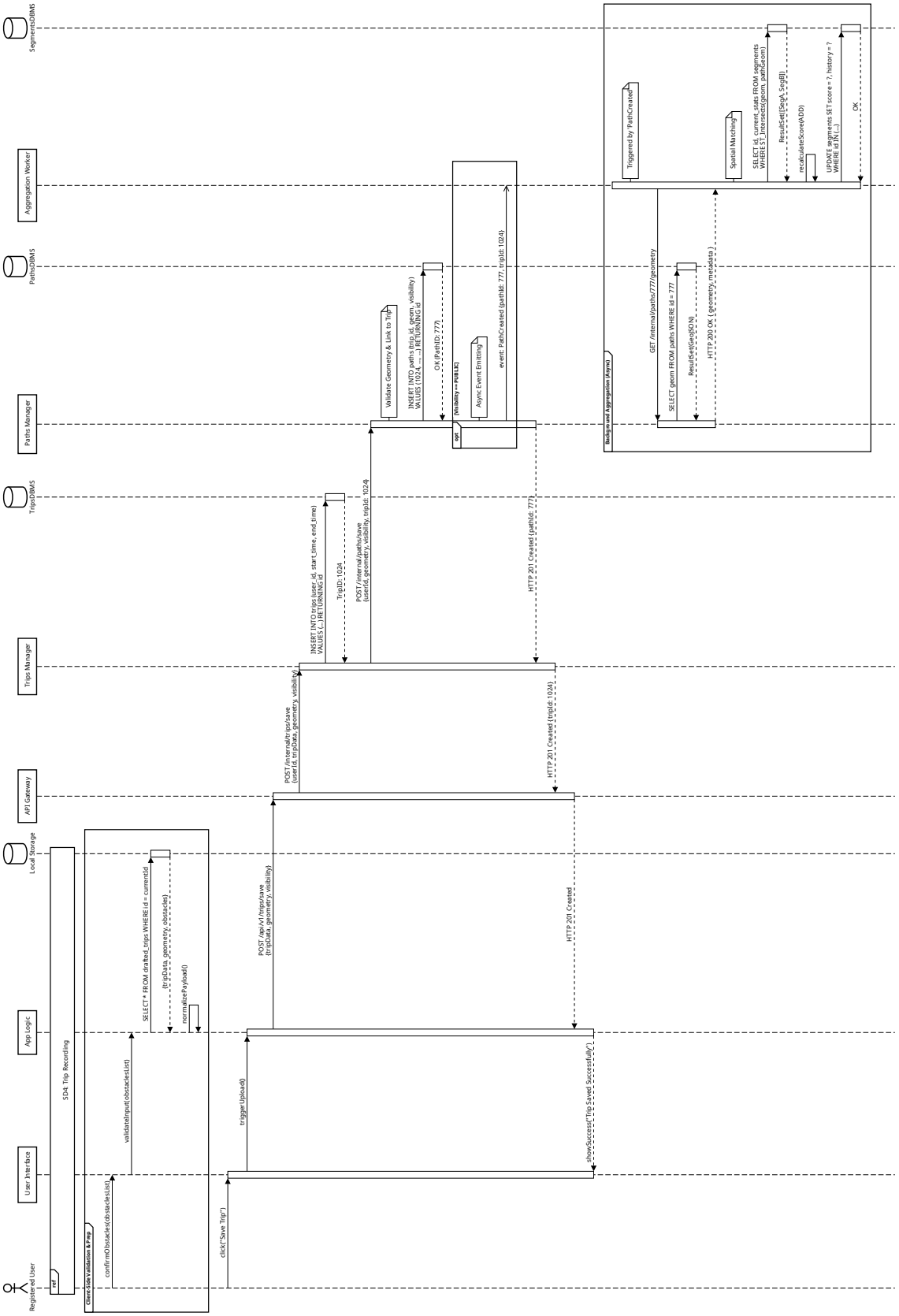


Figure A.9 – SD5. Full page diagram

A

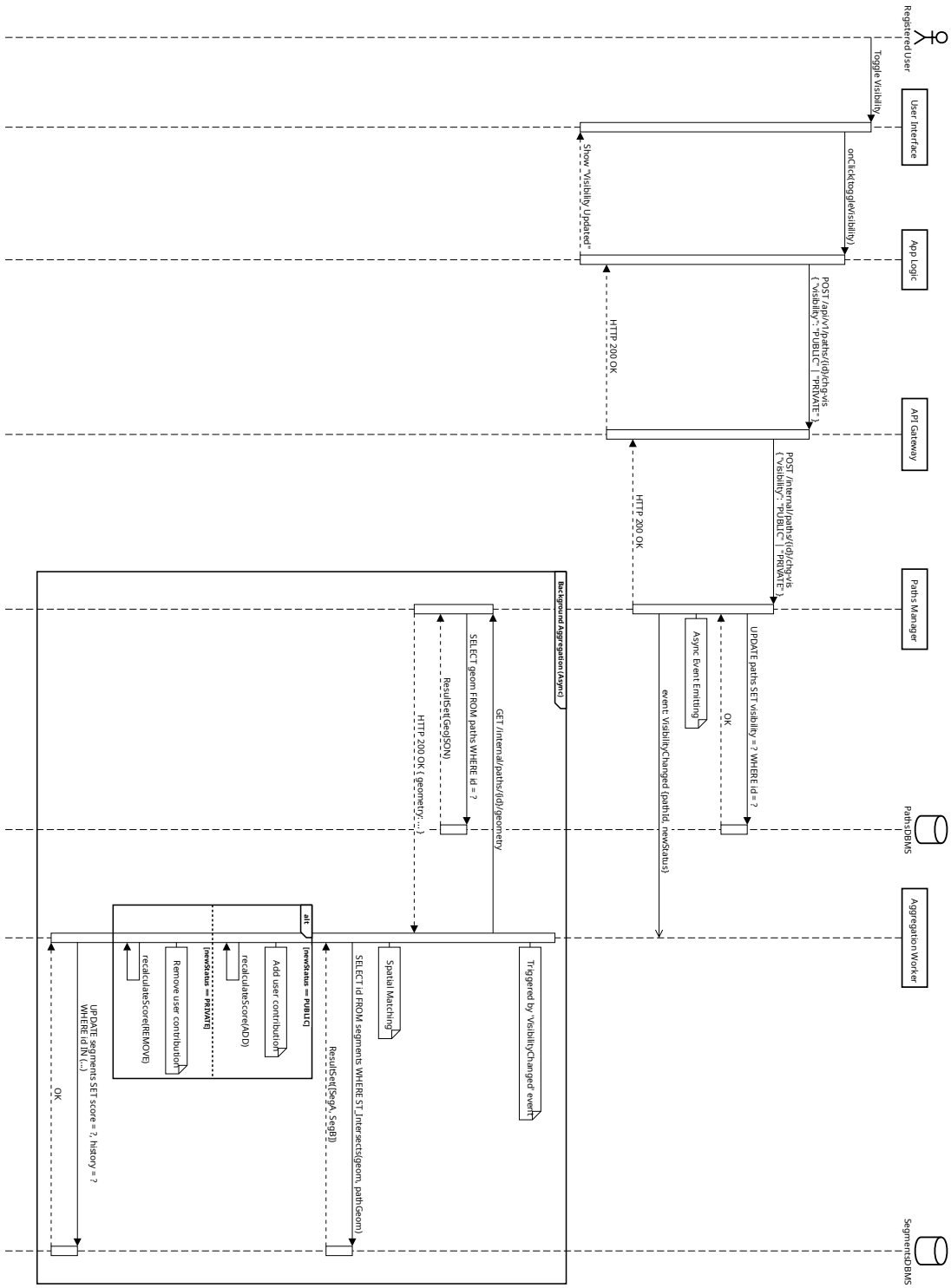


Figure A.10 – SD6. Full page diagram

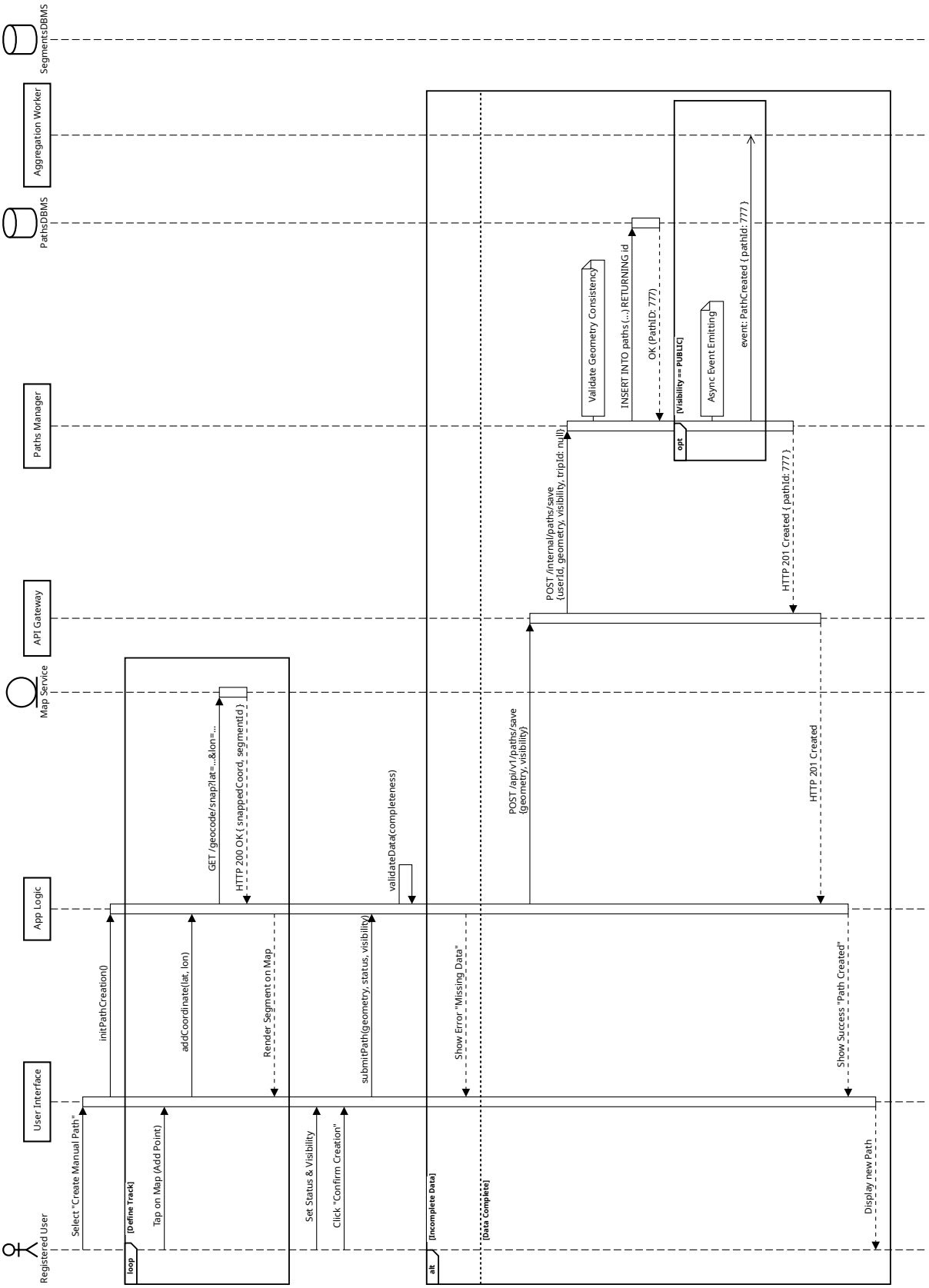


Figure A.11 – SD7. Full page diagram (part 1)

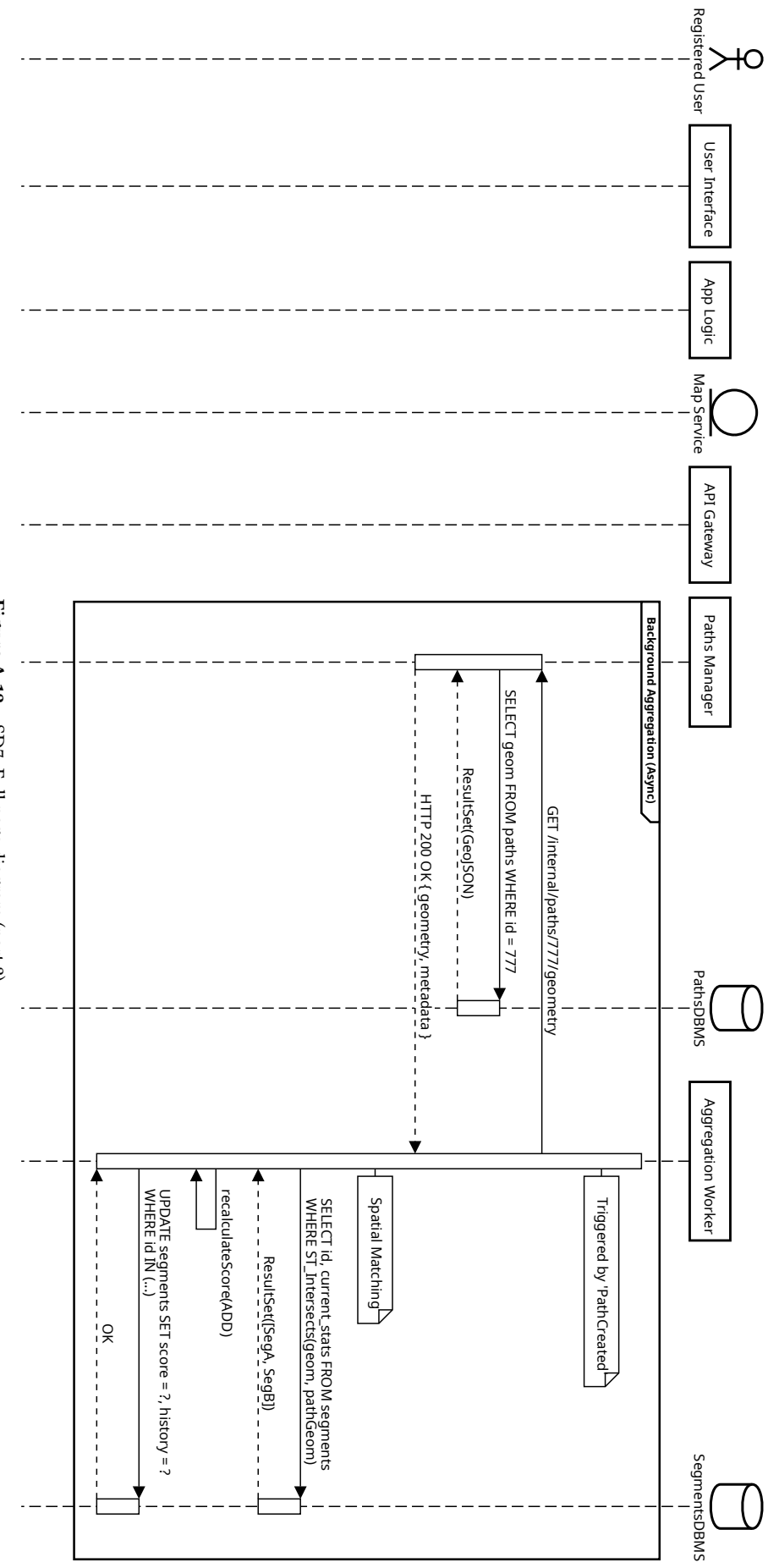


Figure A.12 – SD7. Full page diagram (part 2)

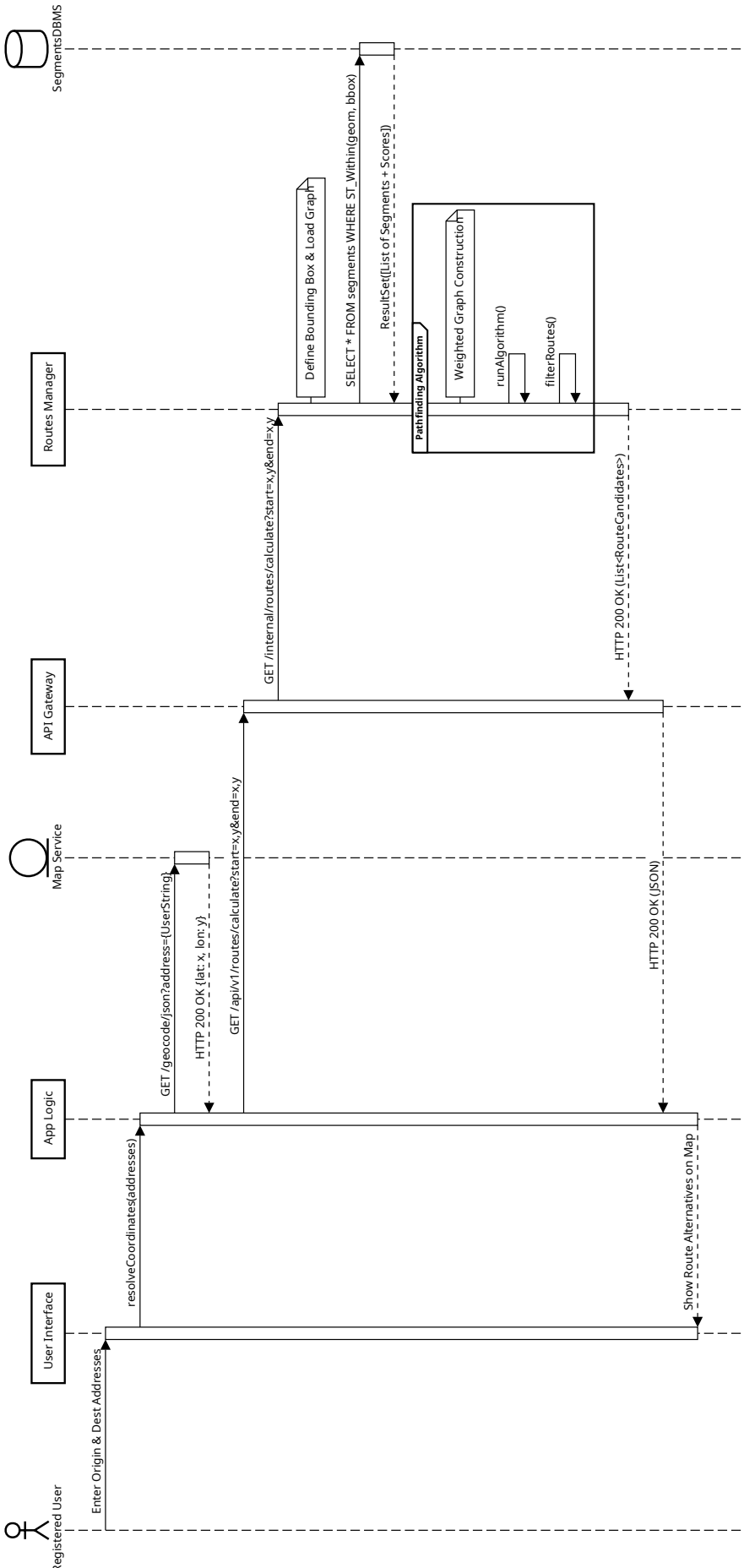


Figure A.13 – SD8. Full page diagram

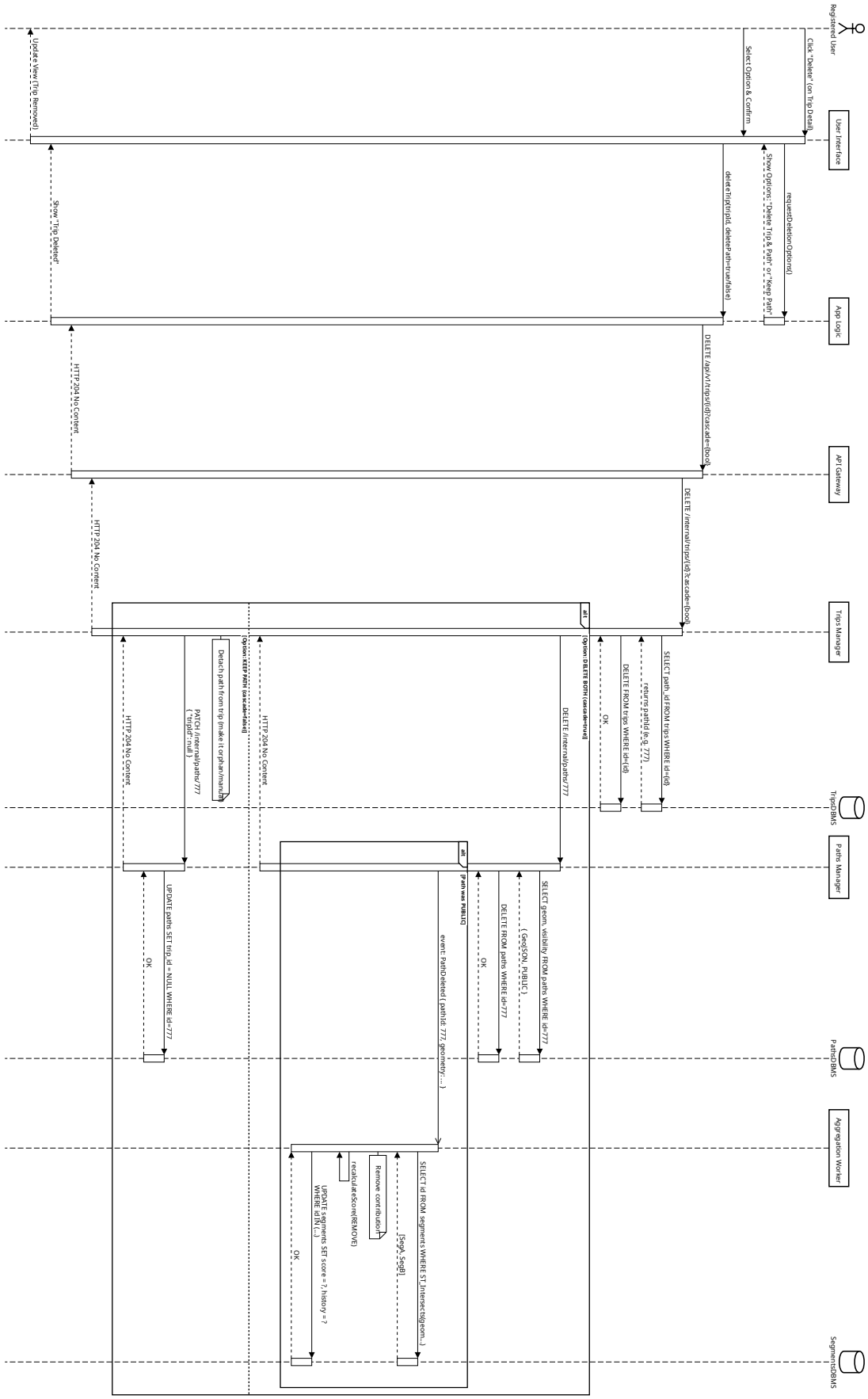


Figure A.14 – SD9. Full page diagram

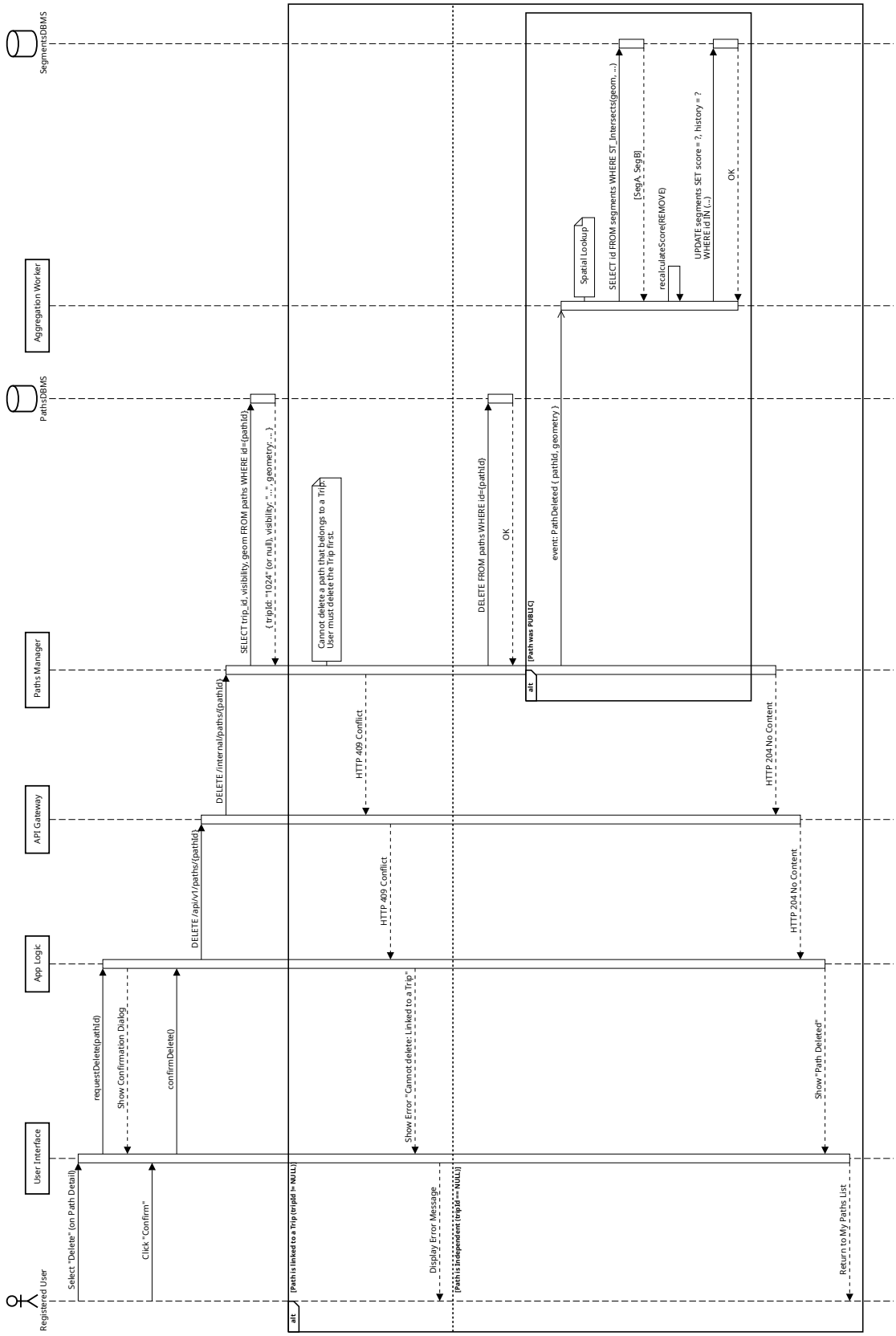


Figure A.15 – SD10. Full page diagram